

JTAG Debug Mode — Vitis Unified IDE

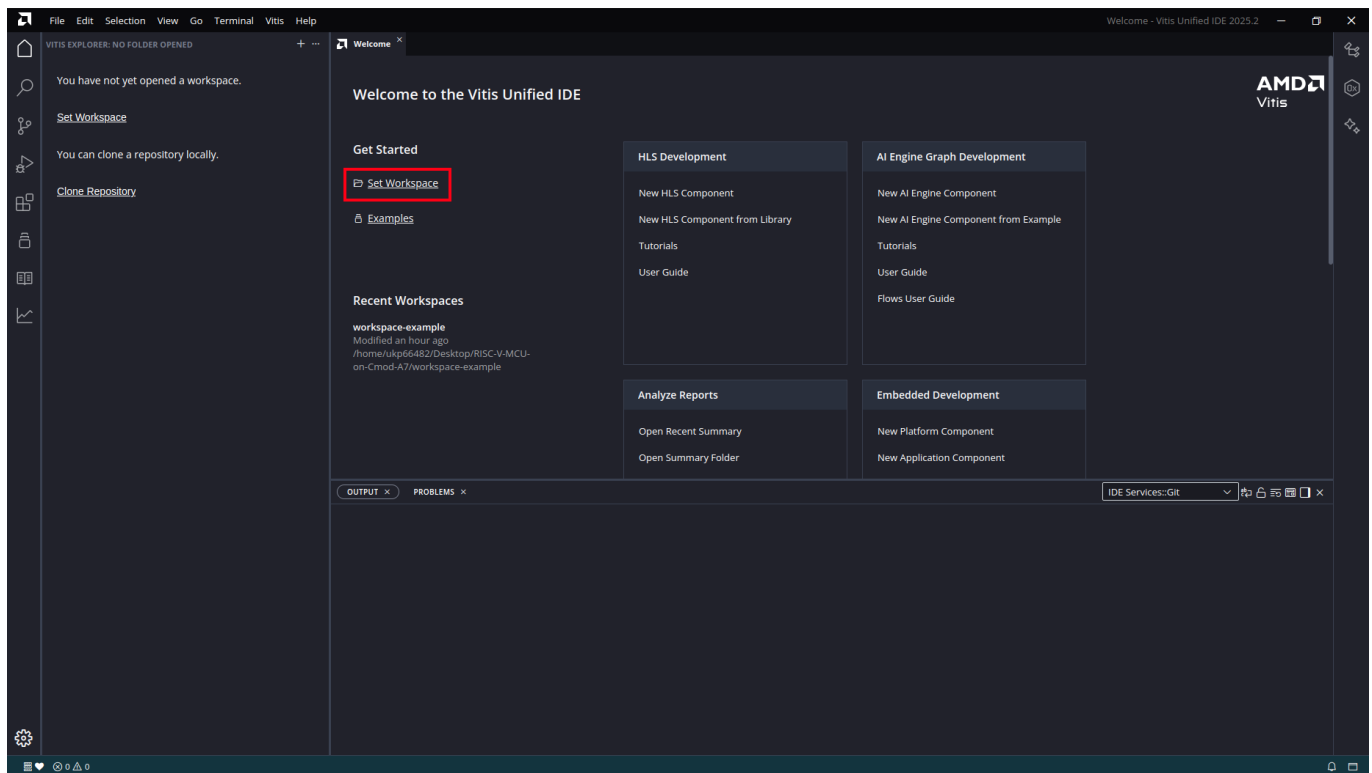
This guide describes how to load and debug a MicroBlaze RISC-V application over JTAG using the AMD Vitis Unified IDE. JTAG loads the program directly into RAM and provides full debug control. Nothing is written to flash.

Prerequisites

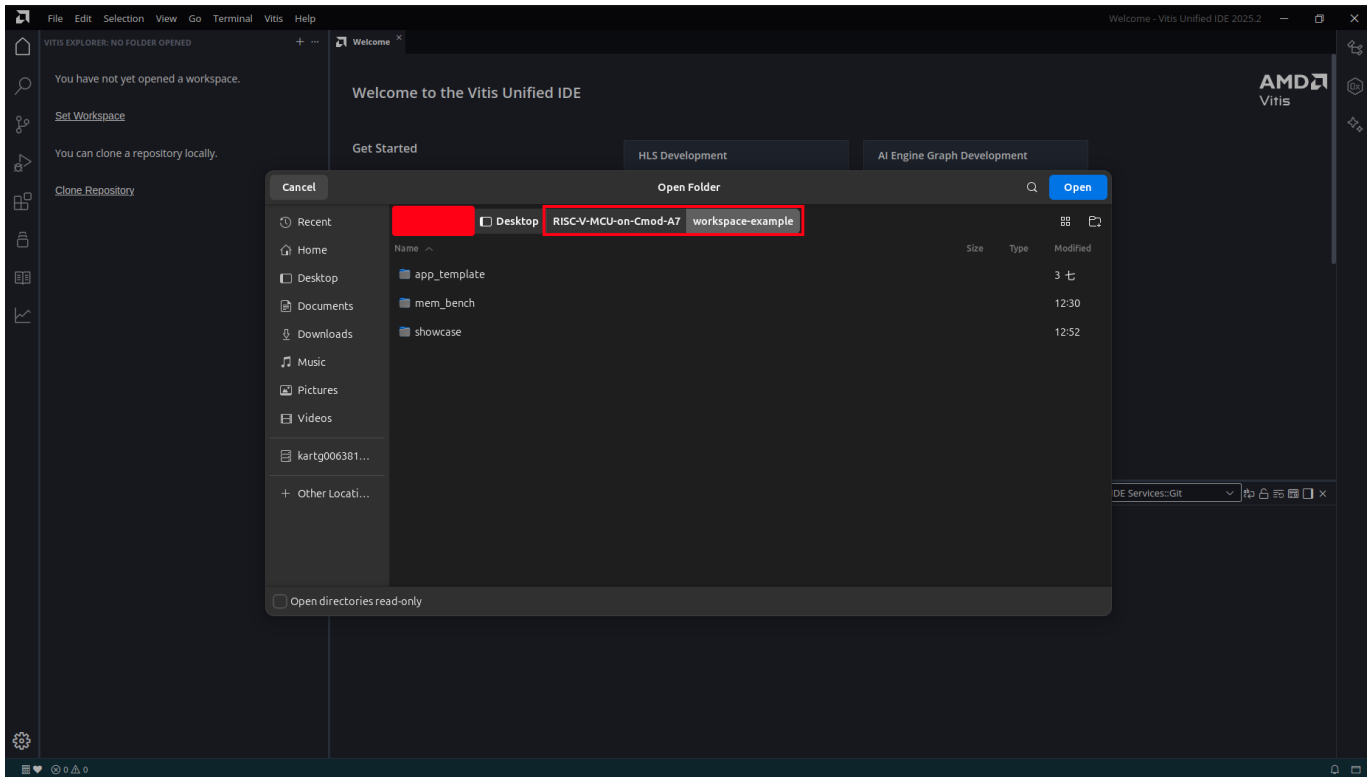
- The hardware design as an `.xsa` file — use the prebuilt `release/top_wrapper.xsa`
- **Vitis Unified IDE 2025.2** installed
- Cmod A7-35T board connected via its micro-USB cable (it carries power, UART, and JTAG all at once)
- A serial terminal program (GTKTerm, PuTTY, or pyserial's miniterm) for the board's console output

1. Open the Workspace

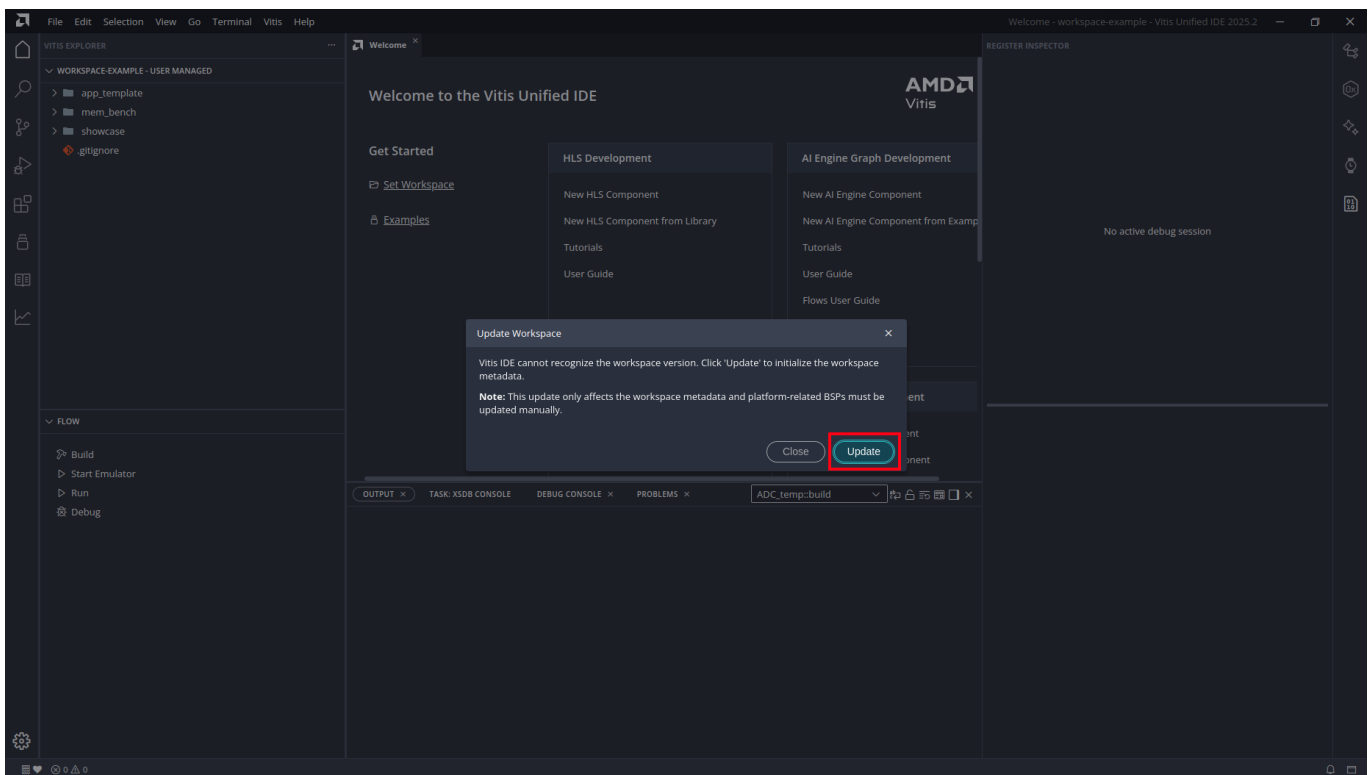
Launch the Vitis Unified IDE and click **Set Workspace** on the Welcome page.



Select the repo's `workspace-example/` folder — the template paths and this guide assume it (any empty folder also works if you adjust the paths yourself).



The first time a Vitis installation opens this workspace it may show an **Update Workspace** dialog. Click **Update** — it refreshes only the workspace metadata and does not modify any component.

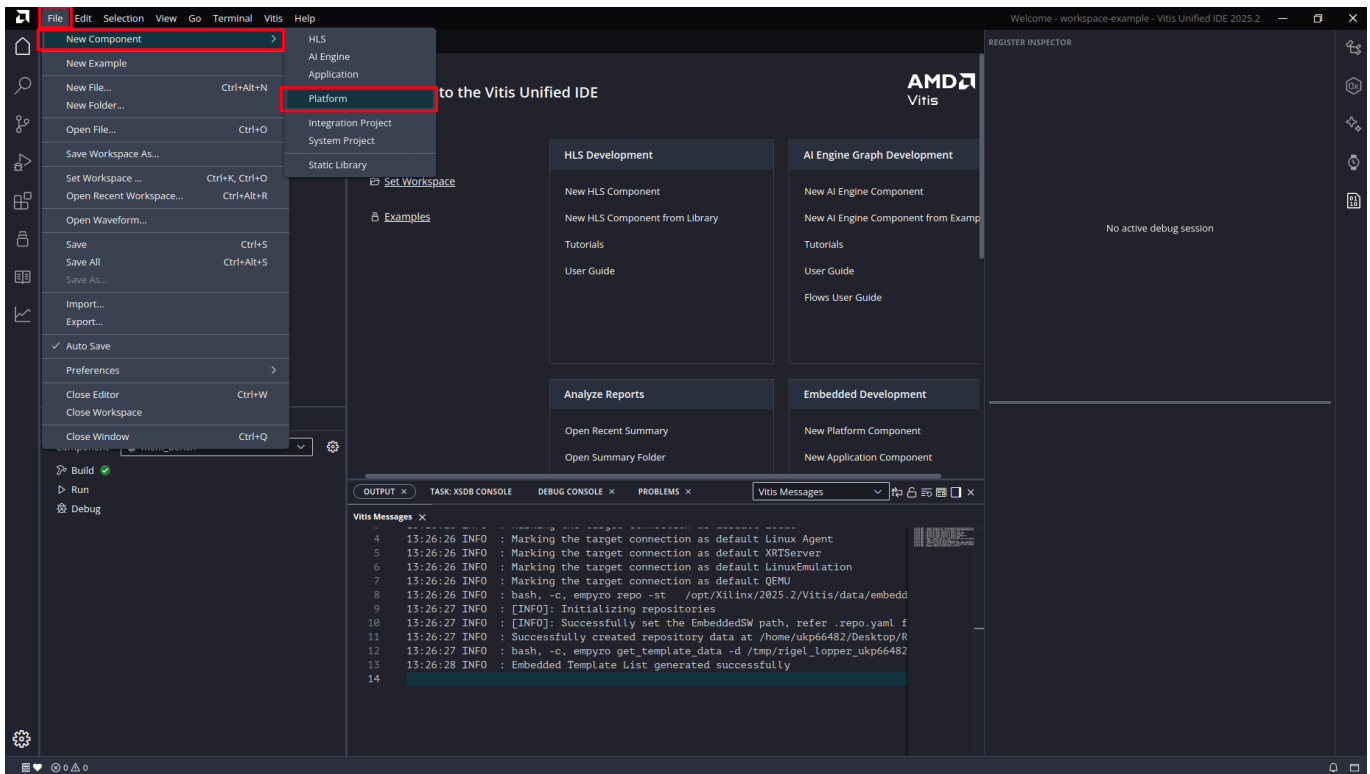


2. Create a Platform

The platform wraps the hardware design and provides the driver layer (BSP). It is created once per workspace and reused by every application. **If the workspace already contains a platform component, skip to section 3.**

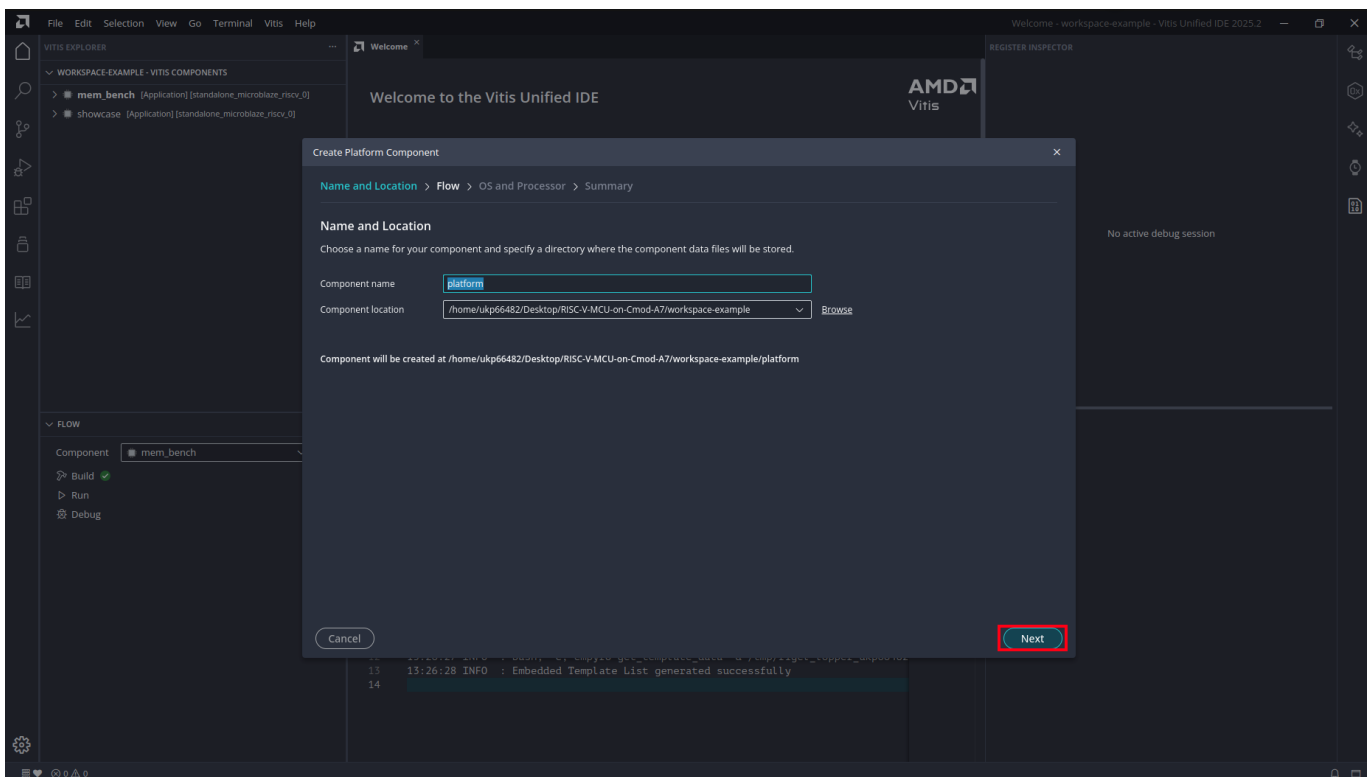
2.1 New Platform Component

From the menu bar, select **File > New Component > Platform**.



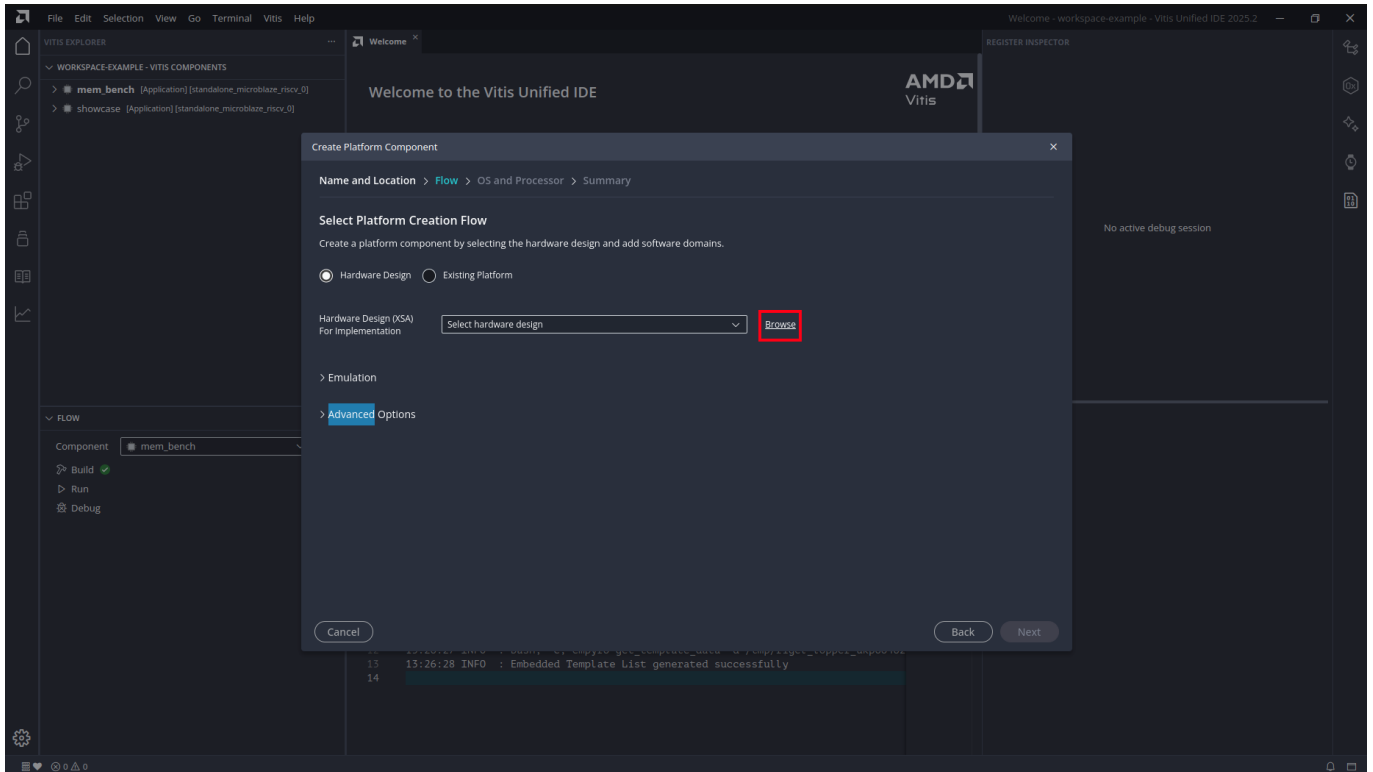
2.2 Name and Location

Enter `platform` as the component name and keep the workspace directory as the location. Click **Next**.

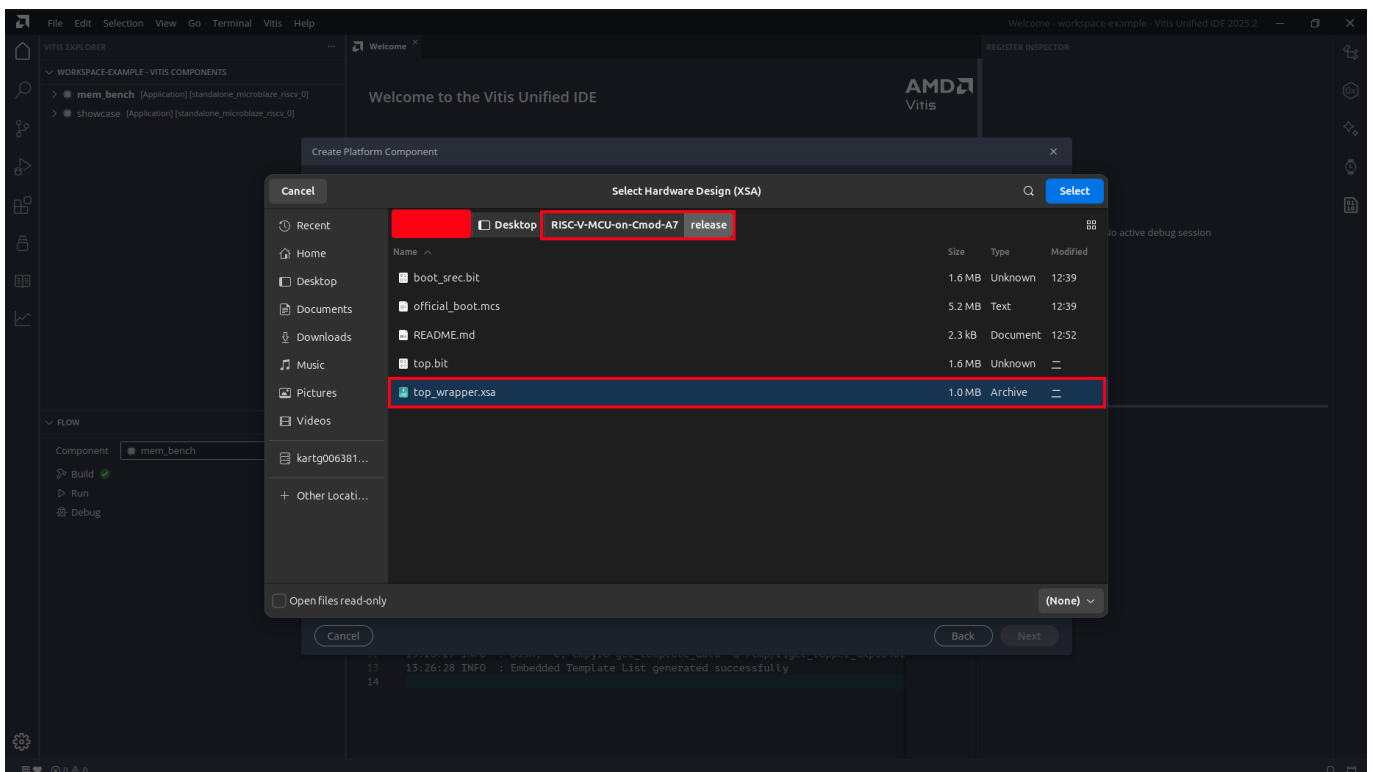


2.3 Select the Hardware Design (XSA)

On the **Flow** page, select **Hardware Design** and click **Browse** next to the *Hardware Design (XSA) For Implementation* field.



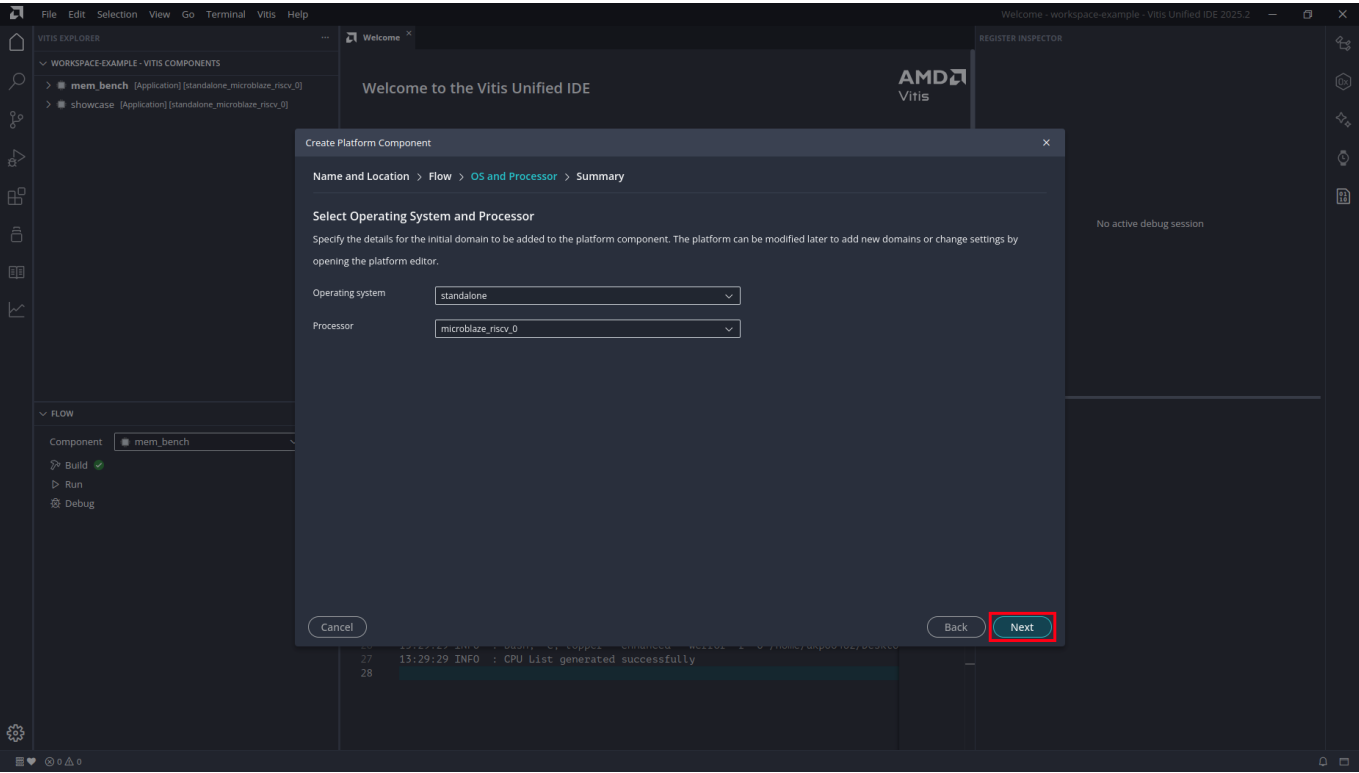
Pick the prebuilt `release/top_wrapper.xsa` from the repository.



2.4 Select Operating System and Processor

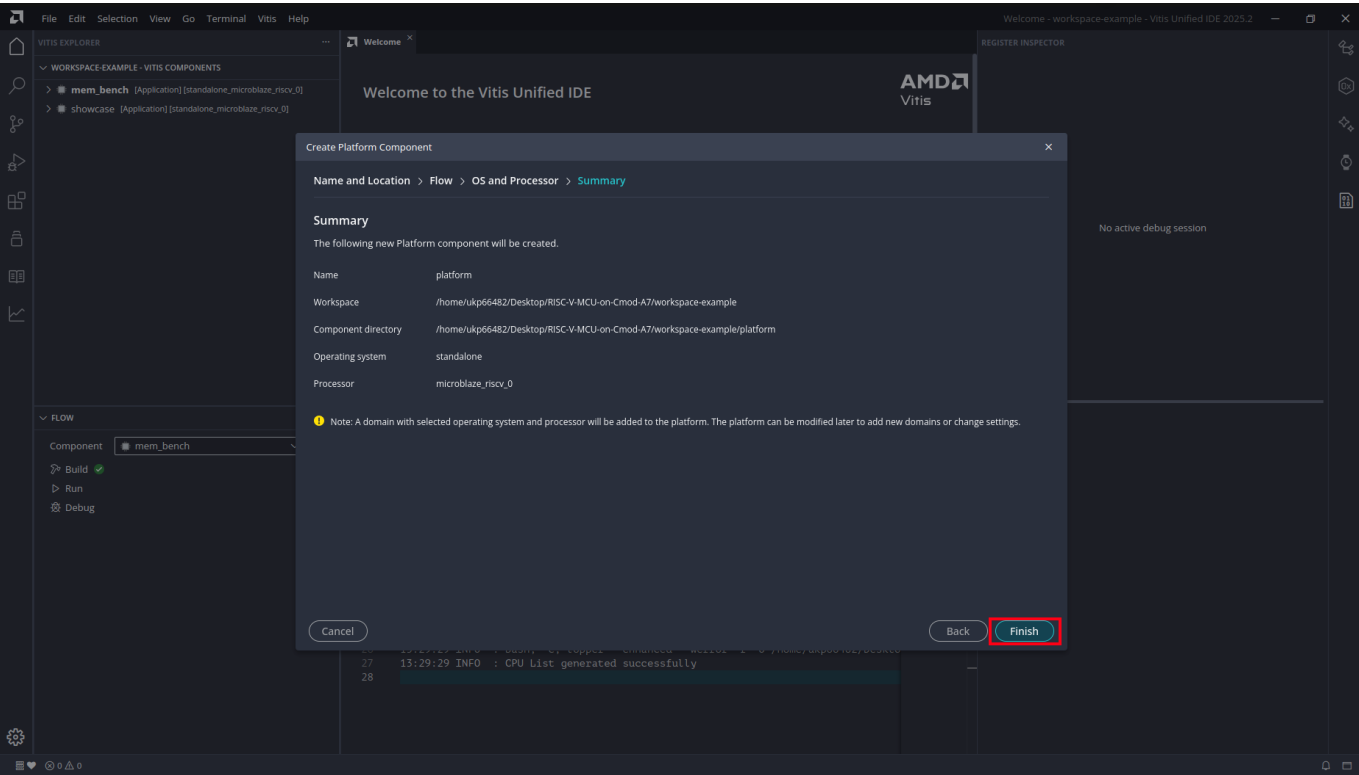
- **Operating system:** standalone
- **Processor:** microblaze_riscv_0

Click **Next**.

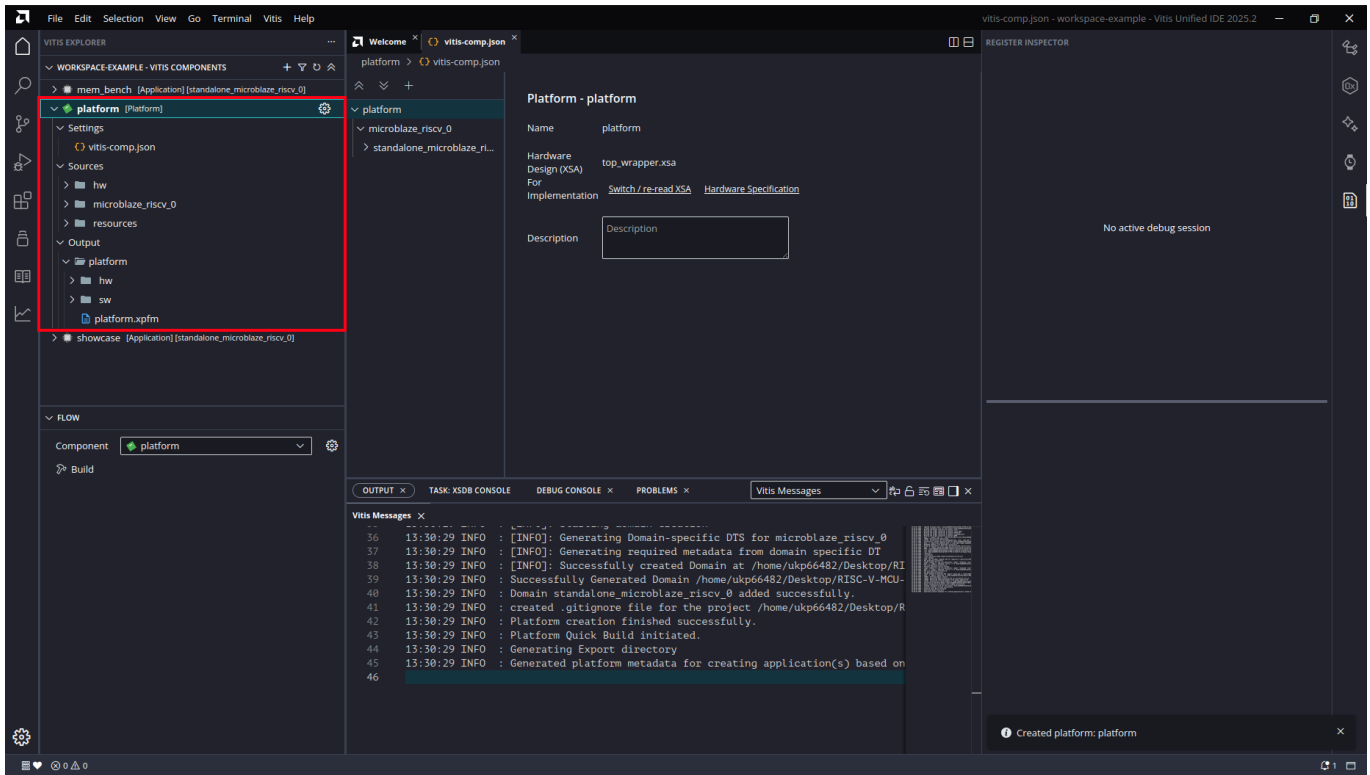


2.5 Summary

Review the settings and click **Finish**.



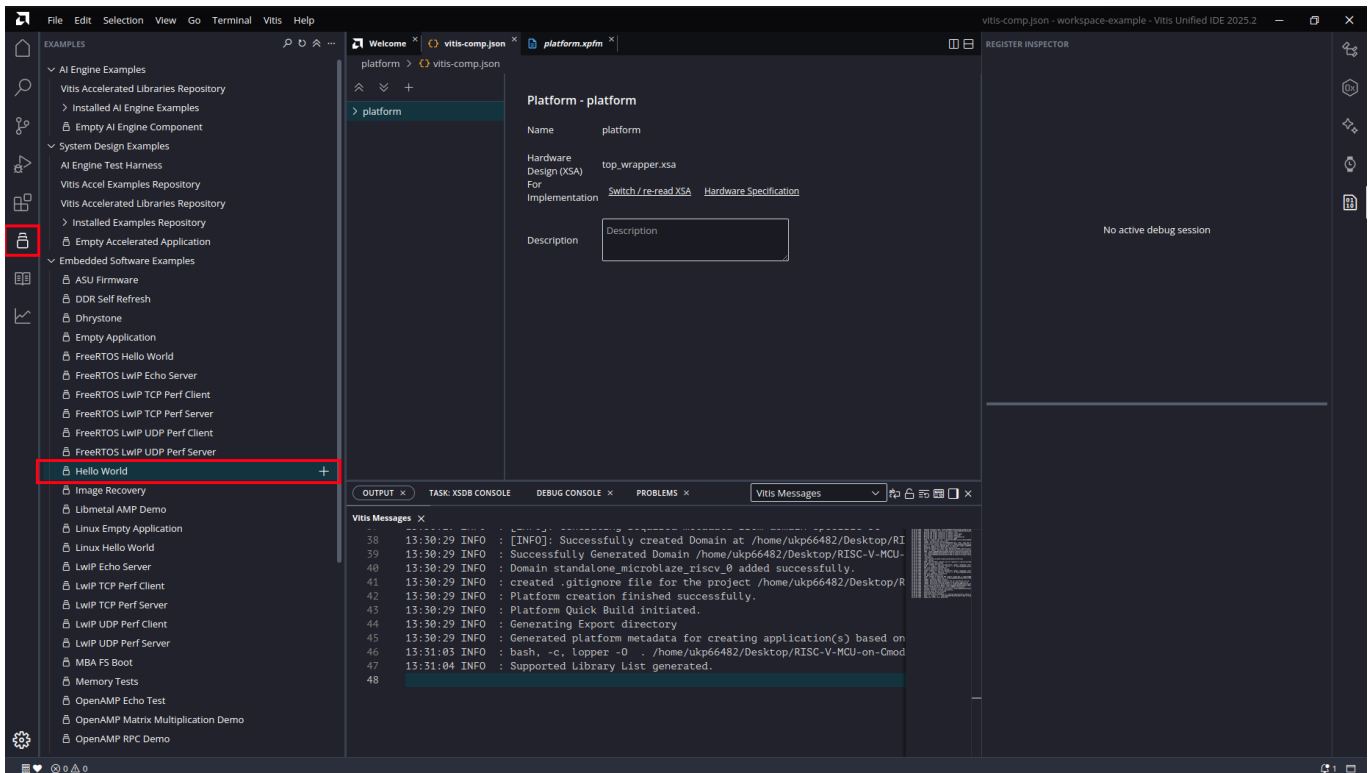
Vitis generates the platform component. When it finishes, the Explorer shows the `platform` component with its `Sources` and `Output` trees, and the notification *"Created platform: platform"* appears.



3. Create an Application

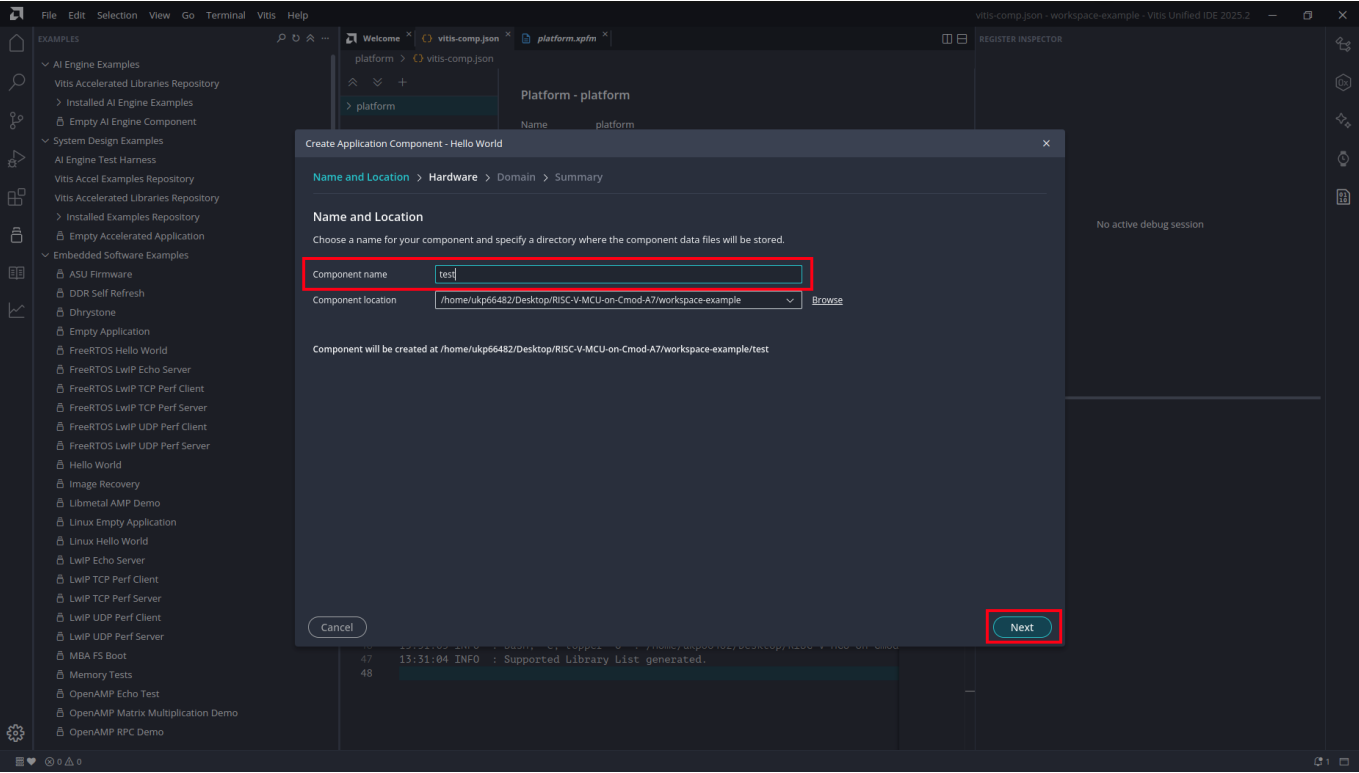
3.1 Create from Template

Open the **Examples** view from the left activity bar. Under *Embedded Software Examples*, select **Hello World** and click the **+** (Create Application Component from Template) button.



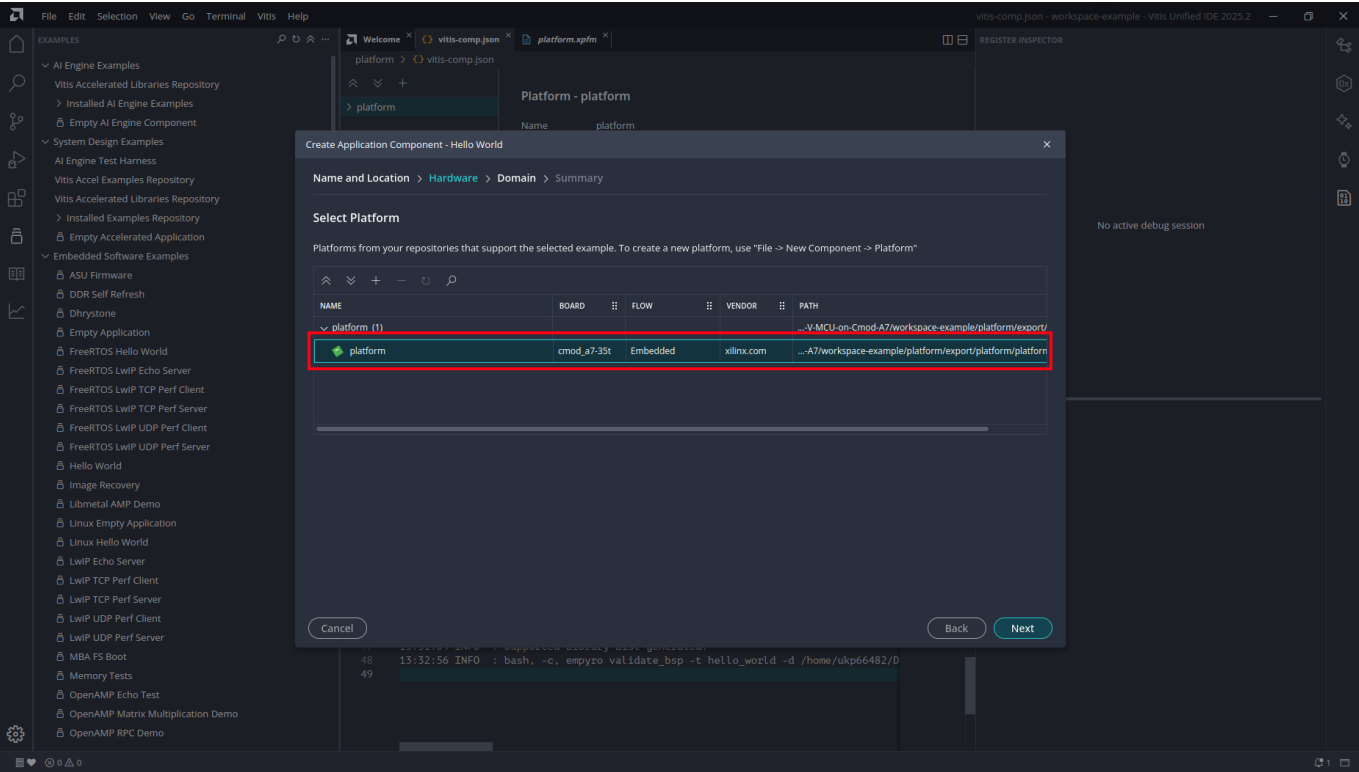
3.2 Name and Location

Enter your application name (this guide uses `test`). Click **Next**.



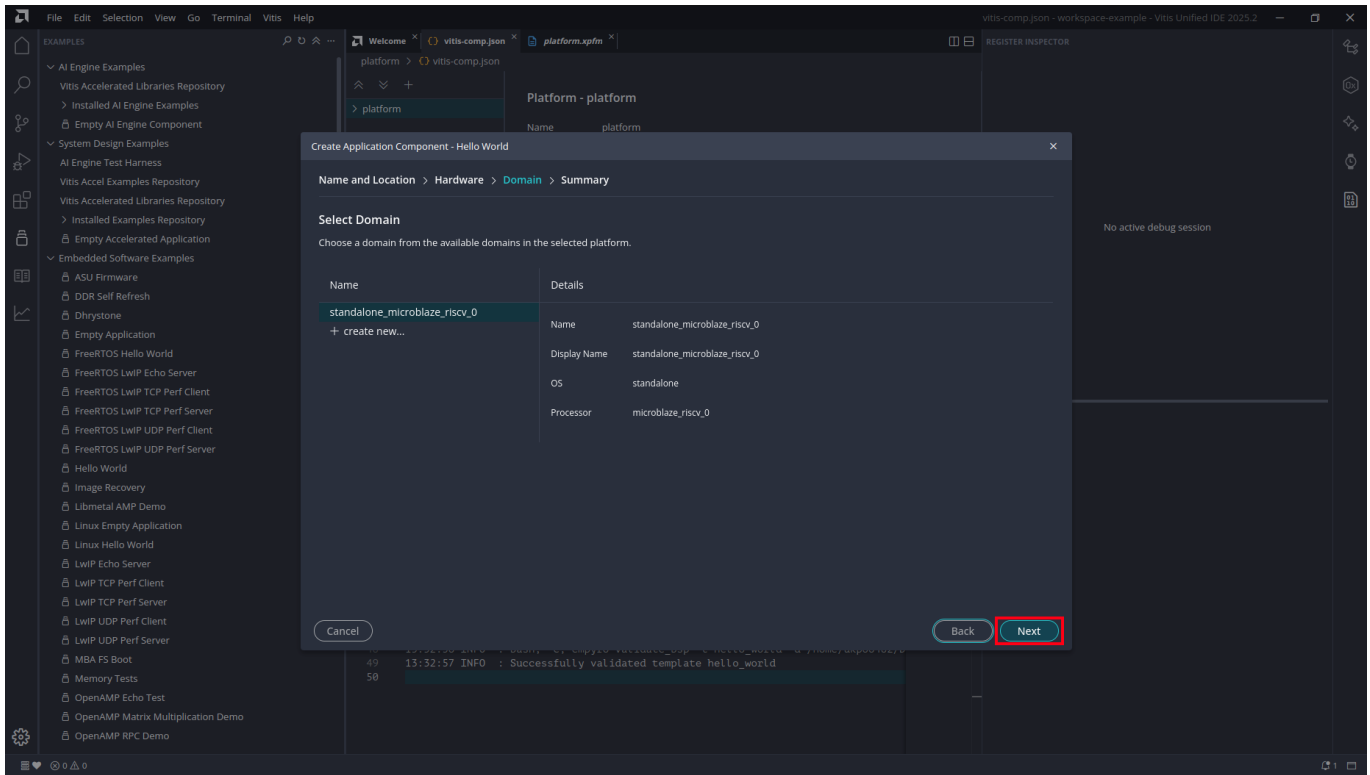
3.3 Select the Platform

Select the `platform` component created in section 2 (Board: `cmod_a7-35t`). Click **Next**.



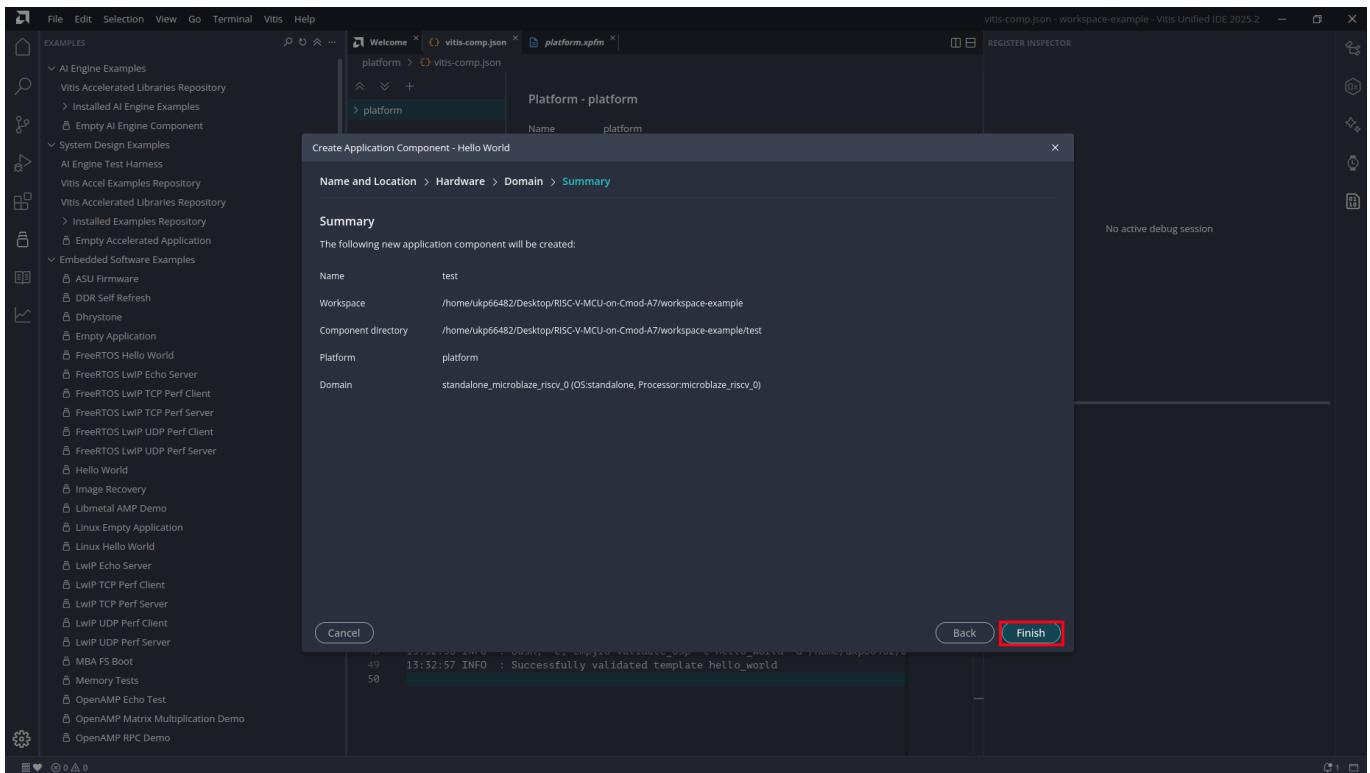
3.4 Select the Domain

Select `standalone_microblaze_riscv_0` . Click **Next**.

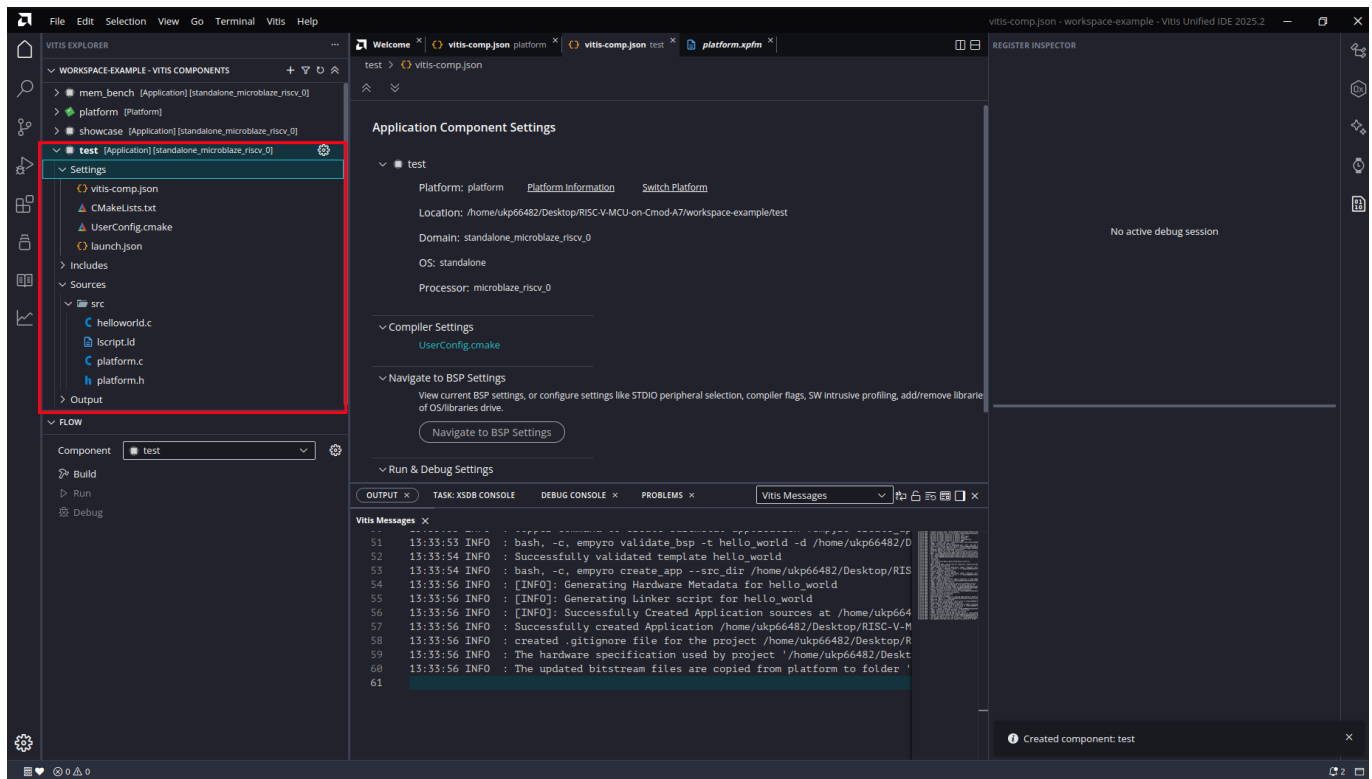


3.5 Summary

Review the settings and click **Finish**.



The new component appears in the Explorer. Its `src/` folder holds the template's generated files: `helloworld.c`, `lscript.ld`, `platform.c`, and `platform.h`.

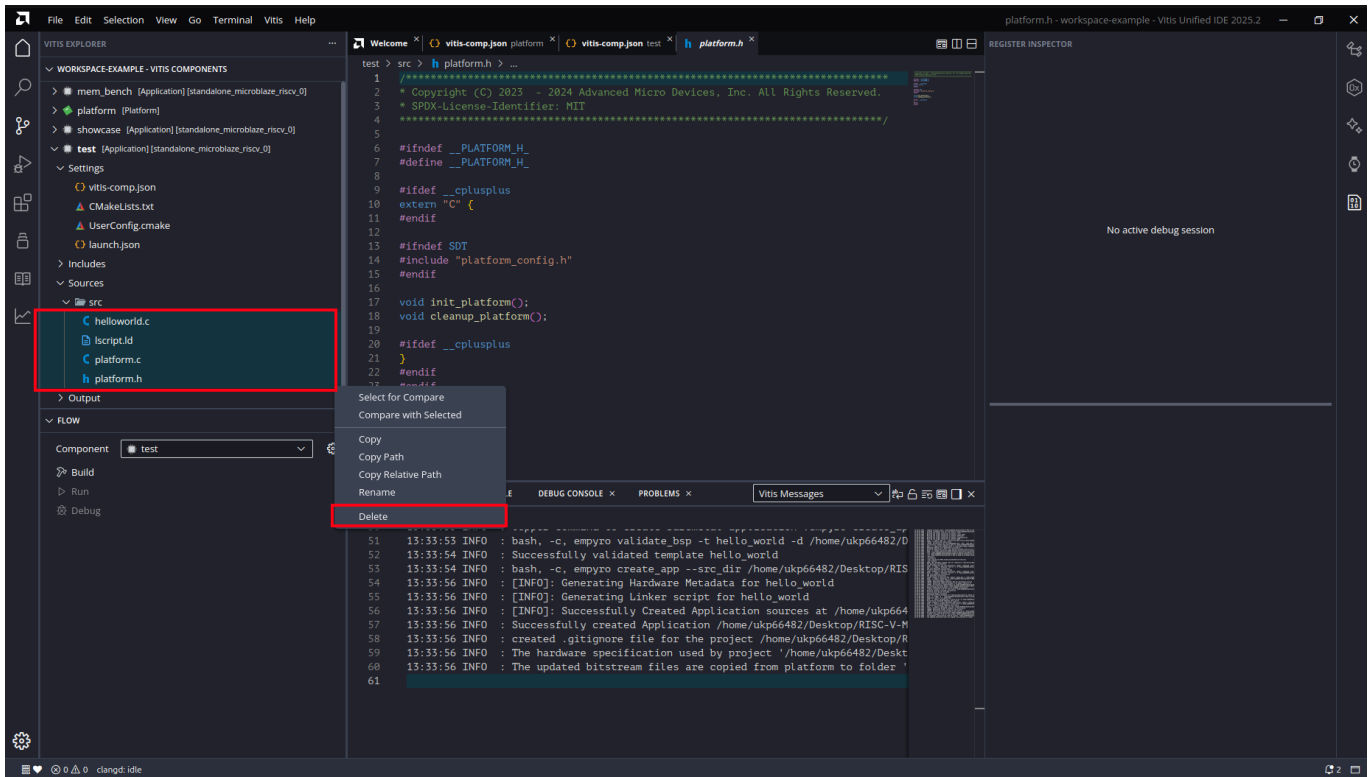


4. Replace the Sources with the Course Template

The generated Hello World sources are replaced entirely by the course template — its `main.c` starts with `mcu_init()` and its `lscript.ld` places code in the 512 KB SRAM, the stack in DTCM, and provides the `ITCM_FUNC / DTCM_DATA` fast-memory attributes.

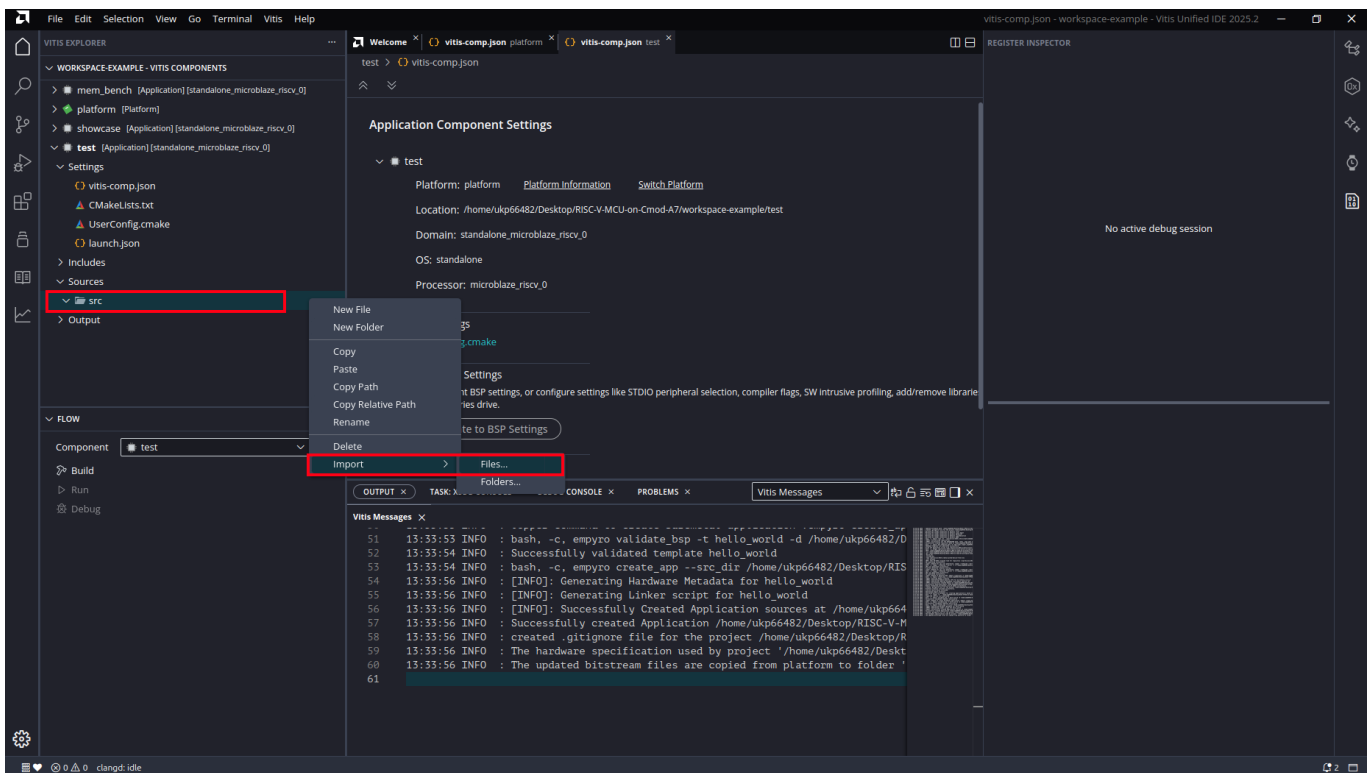
4.1 Delete the Generated Files

In the Explorer, select all four files under `src/` (`helloworld.c`, `lscript.ld`, `platform.c`, `platform.h`), right-click, and choose **Delete**.

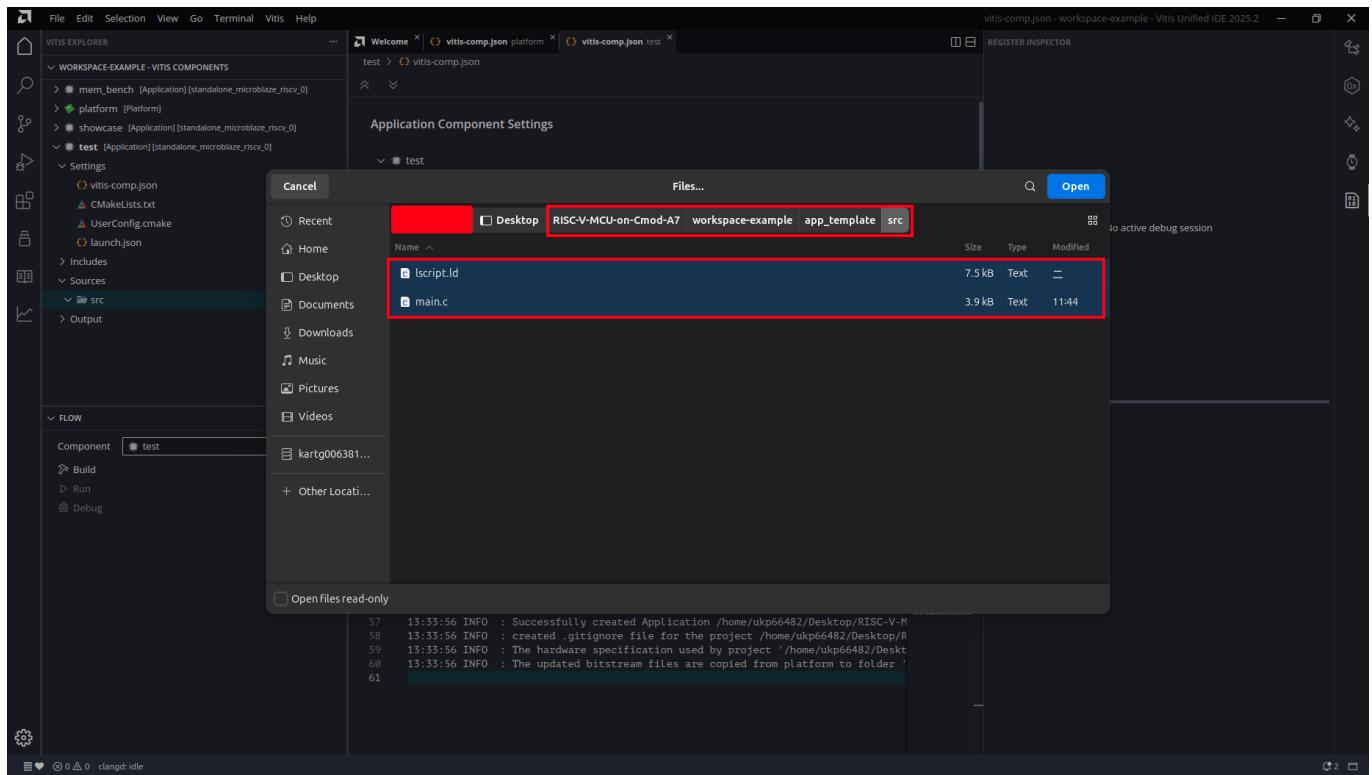


4.2 Import the Template Files

Right-click the `src/` folder and choose **Import > Files...**



Navigate to `workspace-example/app_template/src/` and select both `main.c` and `lscript.ld`. Click **Open**.



Afterwards `src/` contains exactly `main.c` and `lscript.ld`. Files inside `src/` are compiled automatically; no build configuration is needed.

The template's one rule: `mcu_init();` stays the first line of `main()`. It copies `ITCM_FUNC` code into the ITCM, zeroes the DTCM, and sets the USB UART to 115200. The demo loop then cycles the on-board RGB LED and prints a status line about once a second.

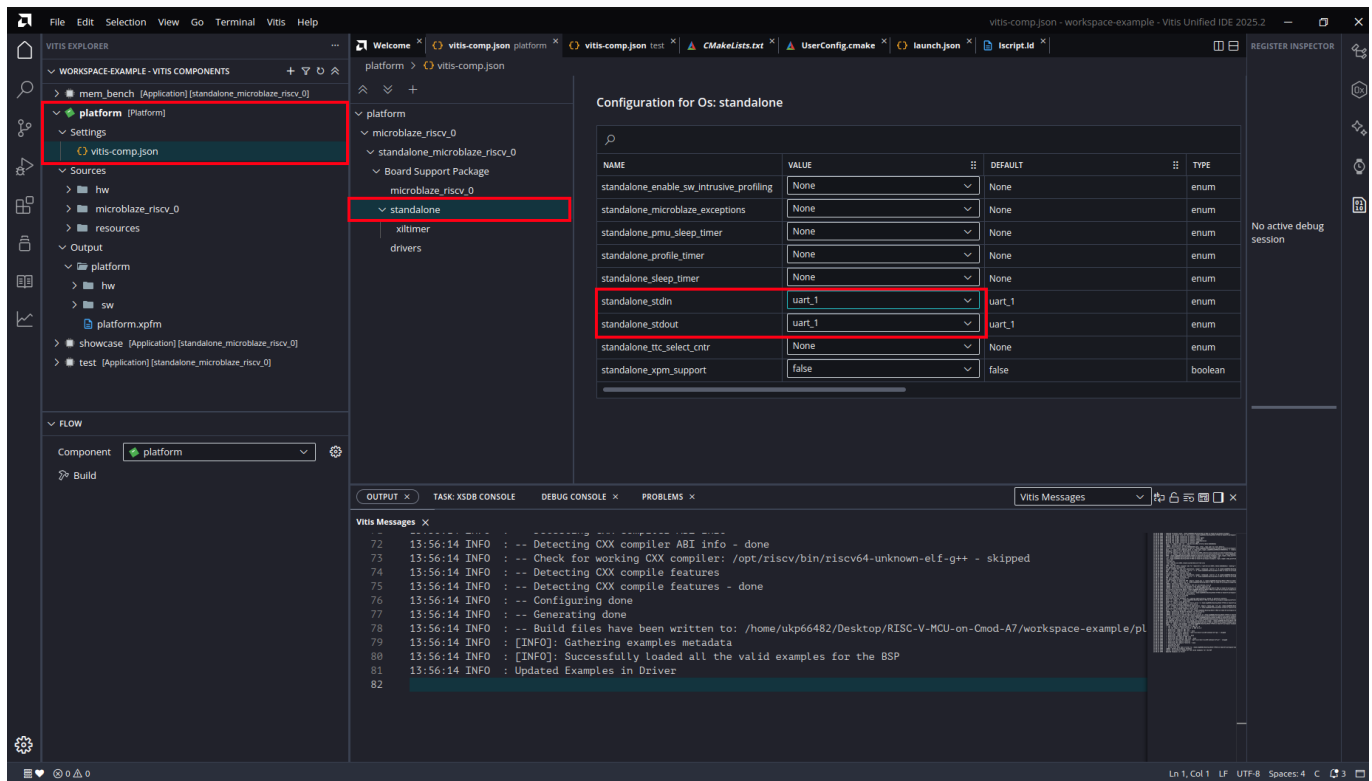
4.3 About the Linker Script

The imported `lscript.ld` defines three memory regions — `sram` (512 KB), `itcm` (32 KB), `dtcm` (64 KB) — and maps everything to SRAM except the stack (DTCM) and `ITCM_FUNC` code (ITCM). The component settings page *Linker Script* can display the regions, but treat it as **read-only**: it understands only part of the hand-written script (the section list may look incomplete — that is expected), and letting it regenerate the script would discard the ITCM/DTCM layout. To resize the stack or heap, edit `_STACK_SIZE` / `_HEAP_SIZE` at the top of `lscript.ld` instead.

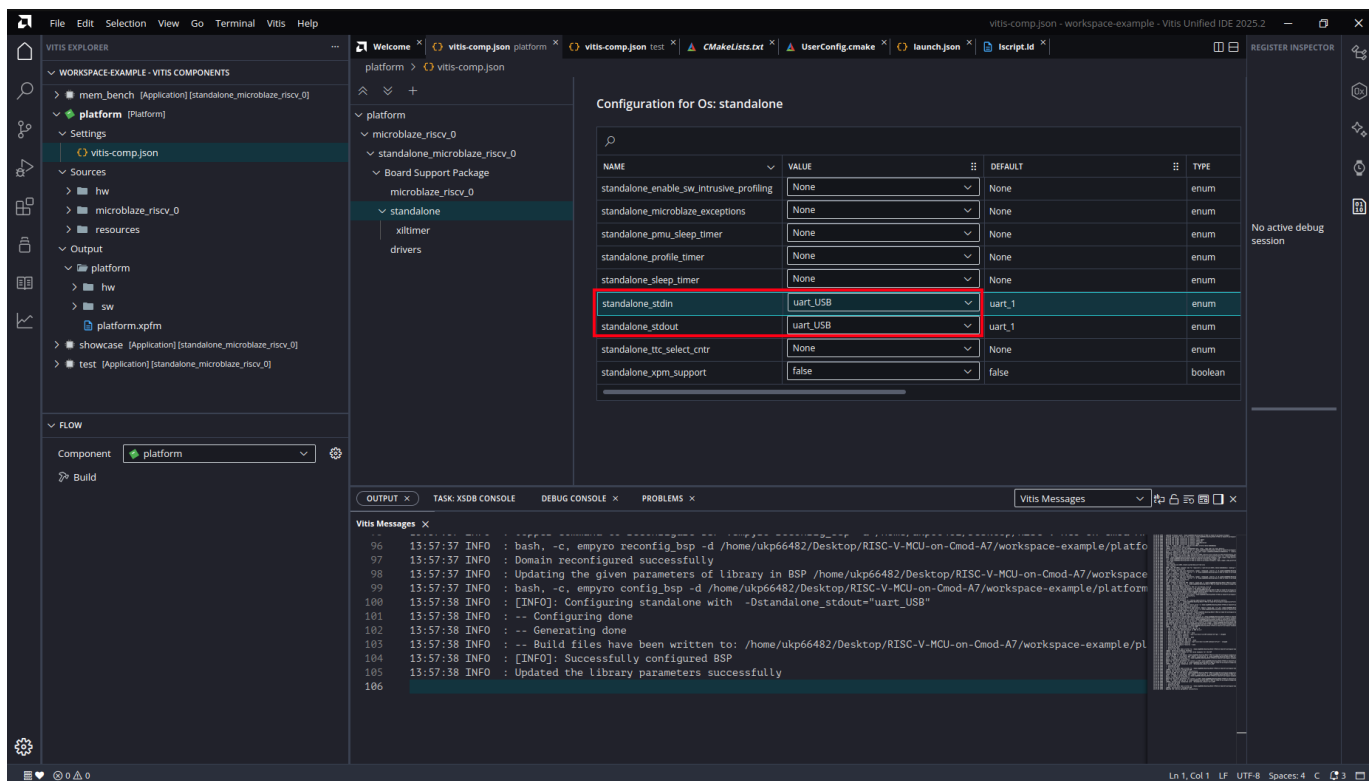
5. Set the Console UART (BSP)

The BSP decides which UART carries `xil_printf` output. The default is `uart_1` — the DIP-header pins — so with it, nothing appears on the USB console.

Open the platform's **Settings > vitis-comp.json**, navigate to **standalone** (Board Support Package > `microblaze_riscv_0 > standalone`). The configuration table shows `standalone_stdin` and `standalone_stdout` set to `uart_1`:



Change both to **uart_USB**. The output log reports the domain being reconfigured.

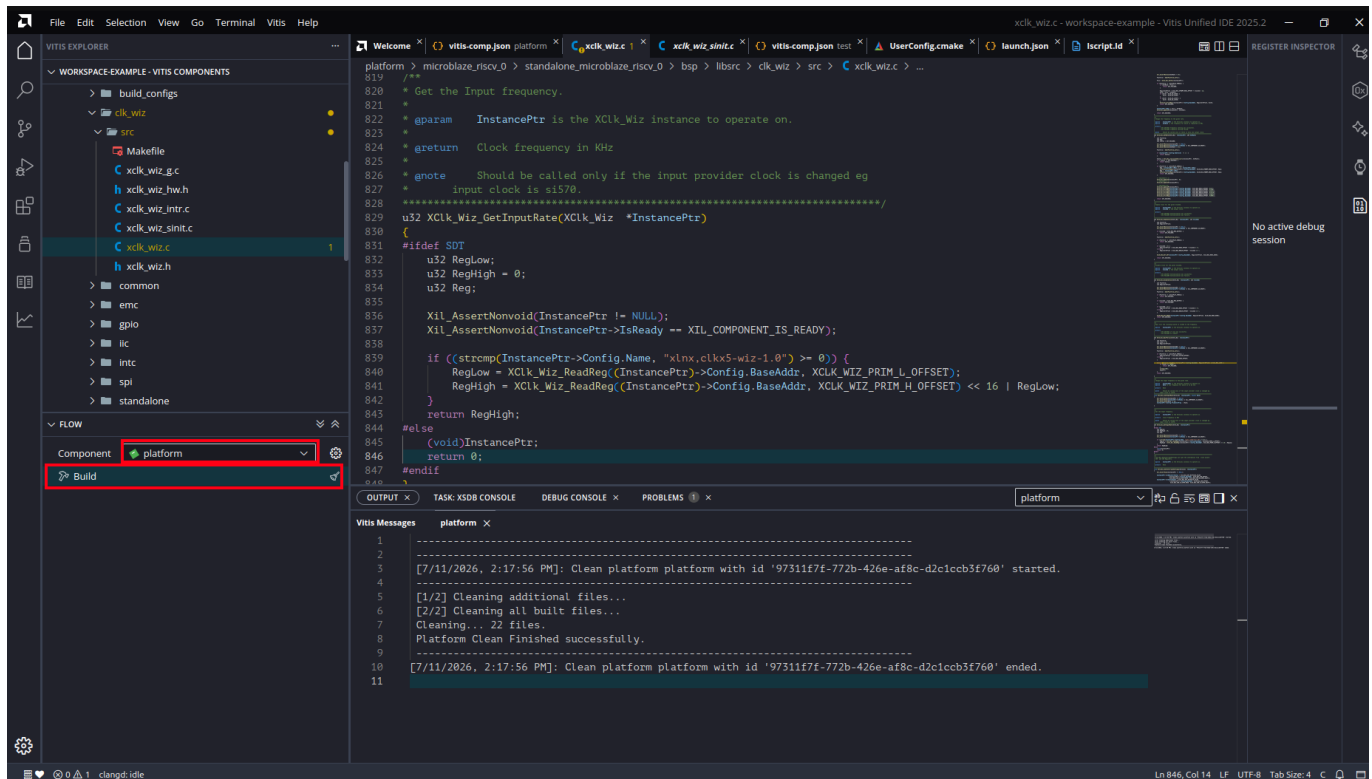


Note: If the platform BSP is ever regenerated (for example after changing another BSP option), re-check this page — regeneration can reset stdin/stdout to **uart_1**. After changing it, rebuild the platform **and** the application; the setting is compiled into the application's BSP library.

6. Build

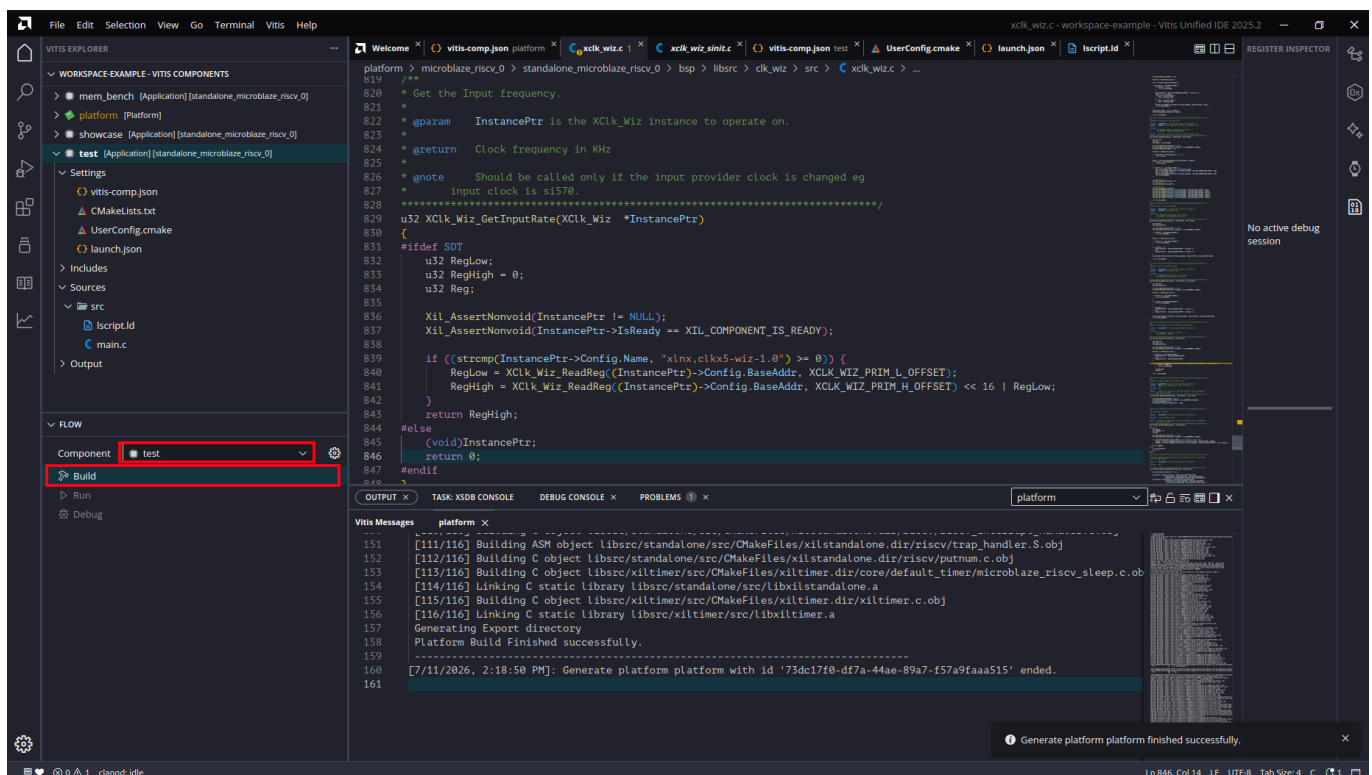
6.1 Build the Platform

In the **FLOW** panel, select the **platform** component and click **Build**. Wait for "*Platform Build Finished successfully*".

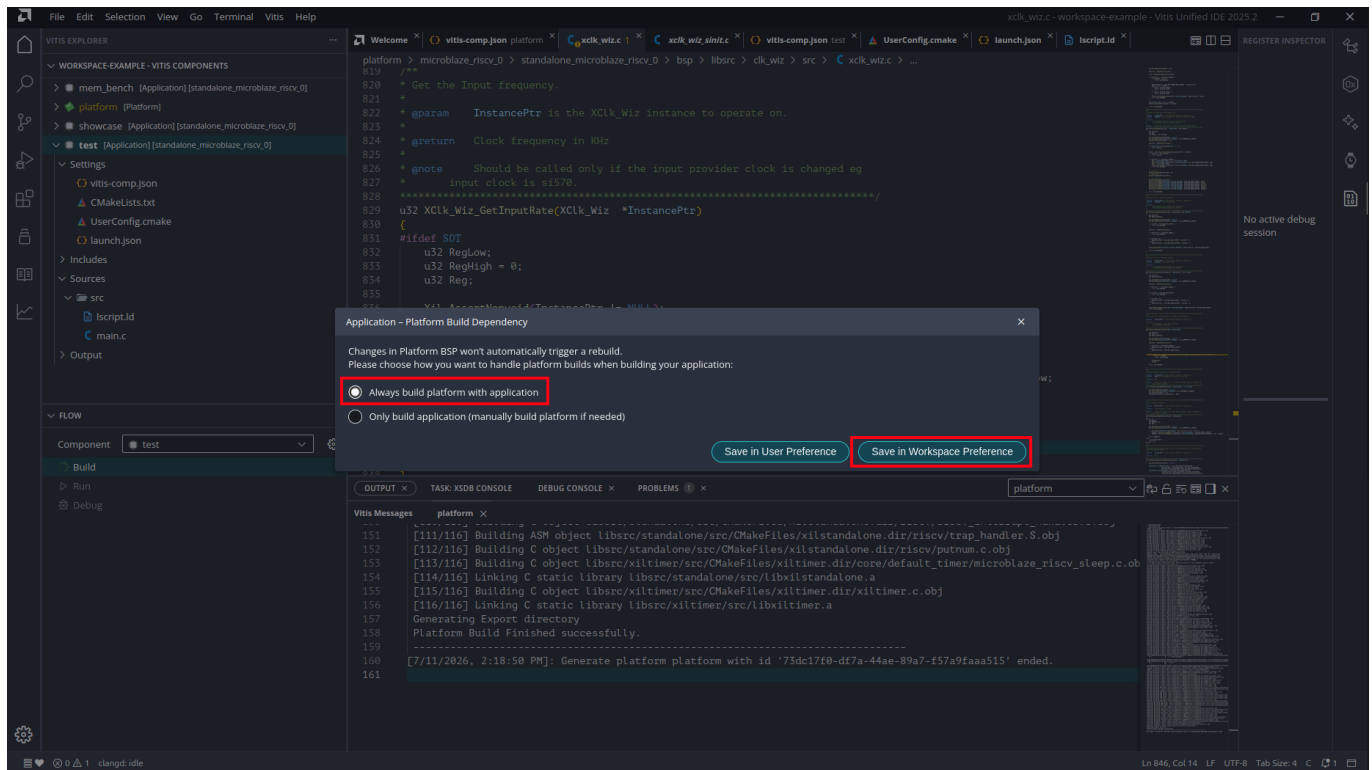


6.2 Build the Application

Switch the FLOW component to your application and click **Build**.



The first application build asks how platform builds should be handled. Select **Always build platform with application** and click **Save in Workspace Preference** — BSP changes (such as section 5) then propagate automatically on every build.



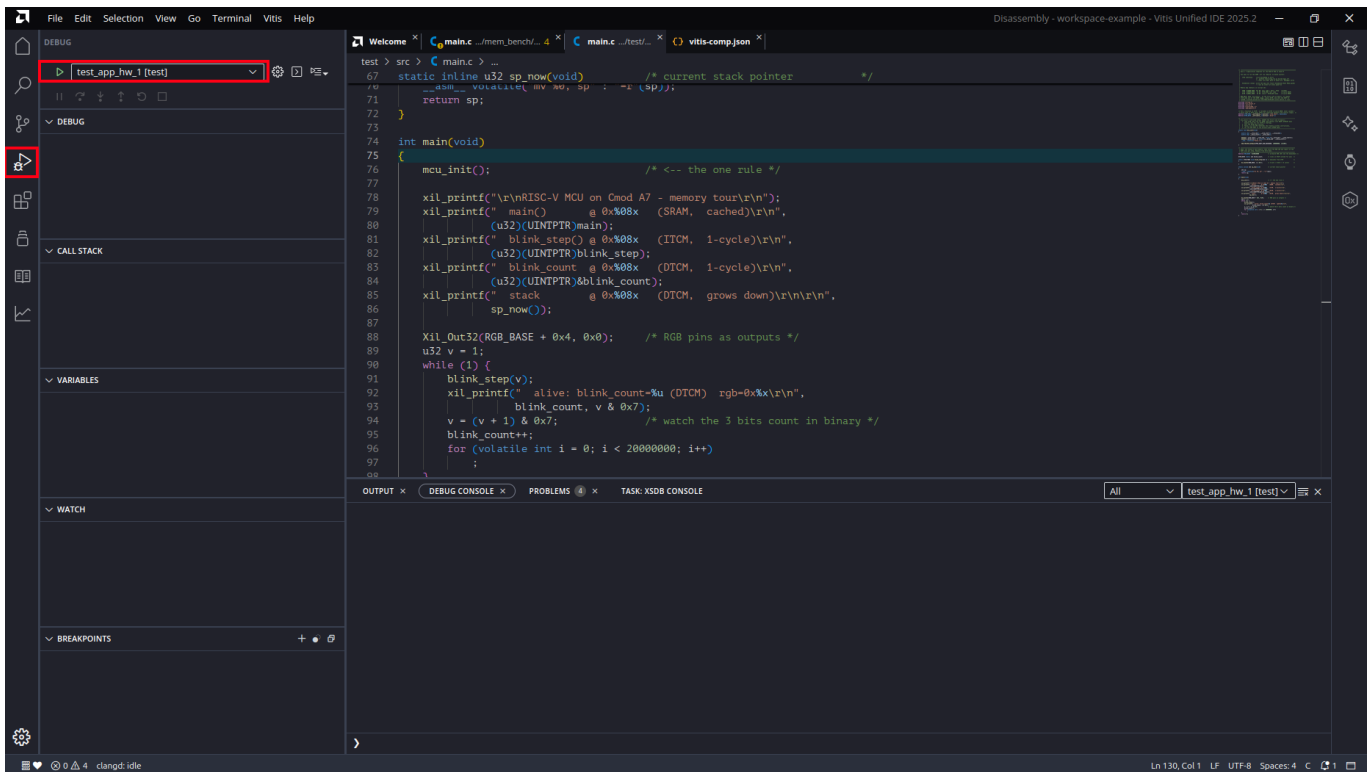
The build produces `<workspace>/test/build/test.elf`.

During the build, Vitis automatically links in `boot.S` and `crt.o` from the standalone BSP: `boot.S` runs first after reset (initializes registers, stack pointer, and trap vector), and `crt.o` clears `.bss` before calling `main()`.

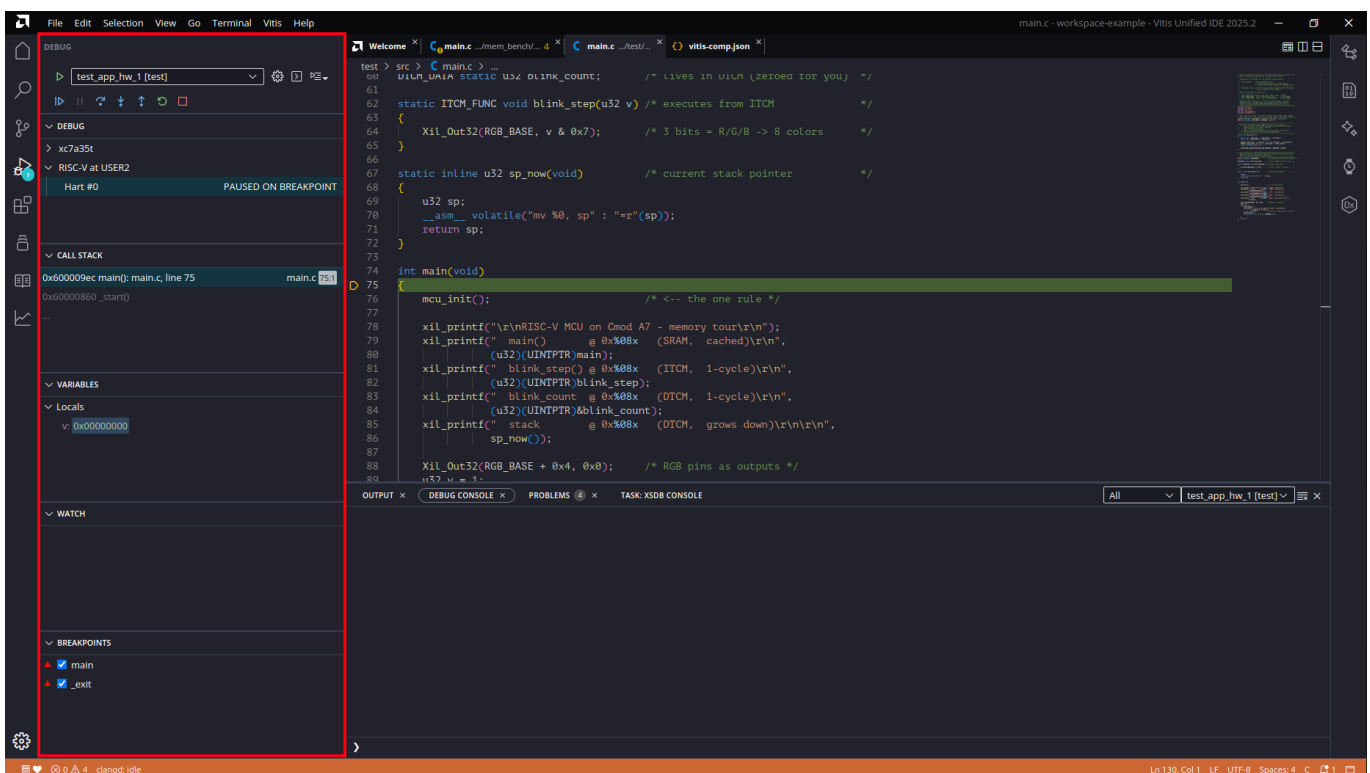
7. Debug over JTAG

7.1 Start a Debug Session

In the **FLOW** panel, click **Debug**. Vitis programs the FPGA over JTAG, loads the ELF into memory, and pauses execution at `main()`.

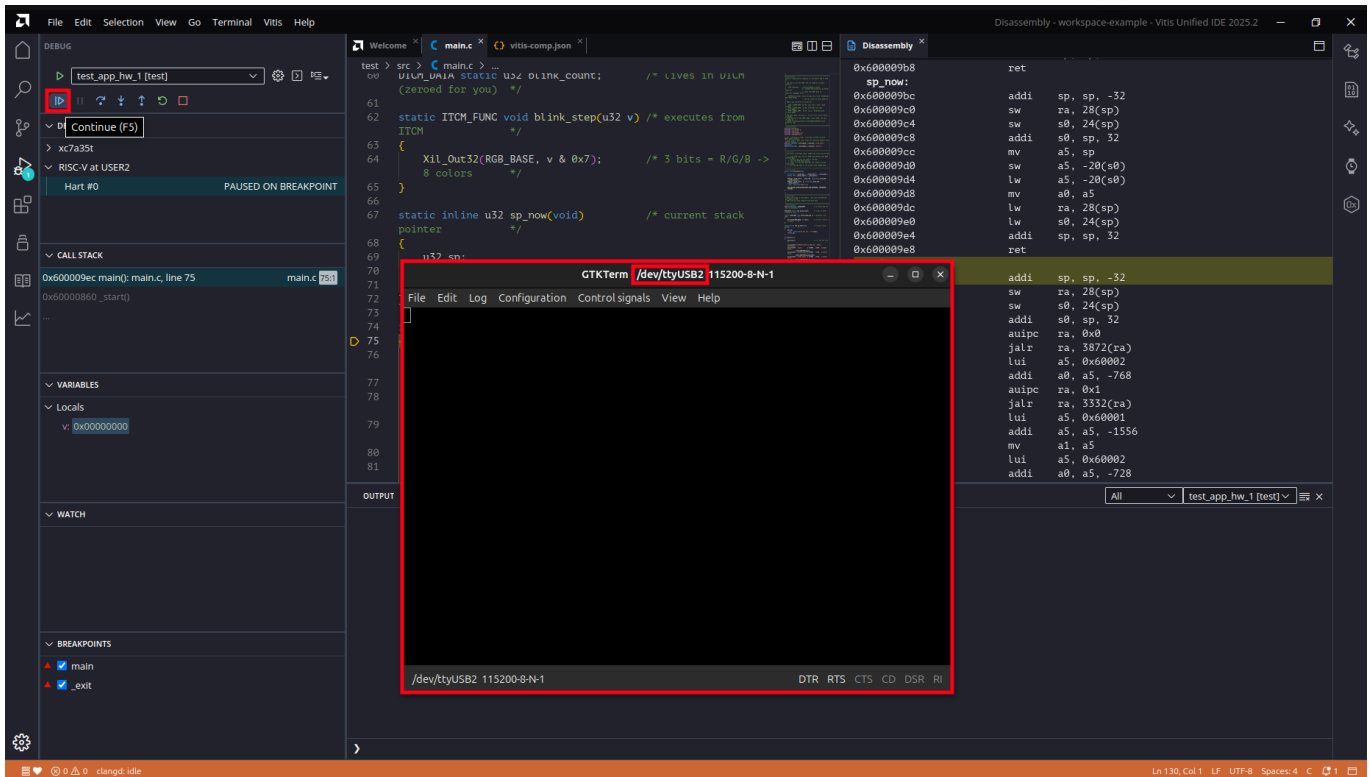


The Debug view shows the target (*RISC-V at USER2, Hart #0 — PAUSED ON BREAKPOINT*), the call stack, local variables, and breakpoints. Note the call stack address: `main()` sits at `0x600009EC` — in SRAM, exactly where the linker script placed it.

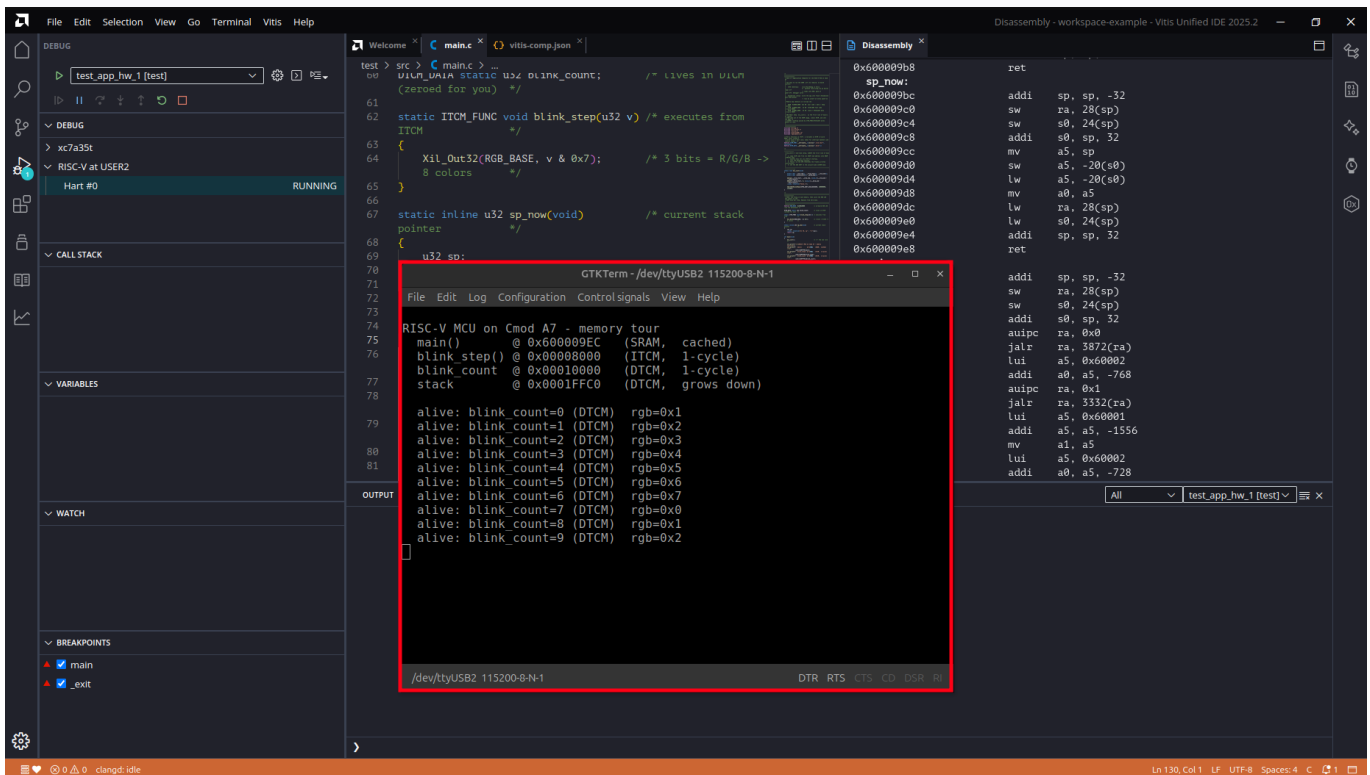


7.2 Open a Serial Terminal, Then Continue

Open a serial terminal **before** resuming execution. The board shows up as two serial ports; the UART is usually the higher-numbered one. Settings: **115200 8N1**.



Click **Continue (F5)**. The application starts: the memory tour prints once, followed by a status line about once a second, and the on-board RGB LED cycles through its colors.



✓ **Checkpoint** — the memory tour is the proof that everything is wired correctly:

```

RISC-V MCU on Cmod A7 - memory tour
main() @ 0x600009EC (SRAM, cached)
blink_step() @ 0x00008000 (ITCM, 1-cycle)

```



```
blink_count @ 0x00010000 (DTCM, 1-cycle)
stack @ 0x0001FFC0 (DTCM, grows down)
```

Each line matches the memory map in the datasheet: code in SRAM, fast code in ITCM, fast data and the stack in DTCM. If the terminal stays silent, see the Troubleshooting section.

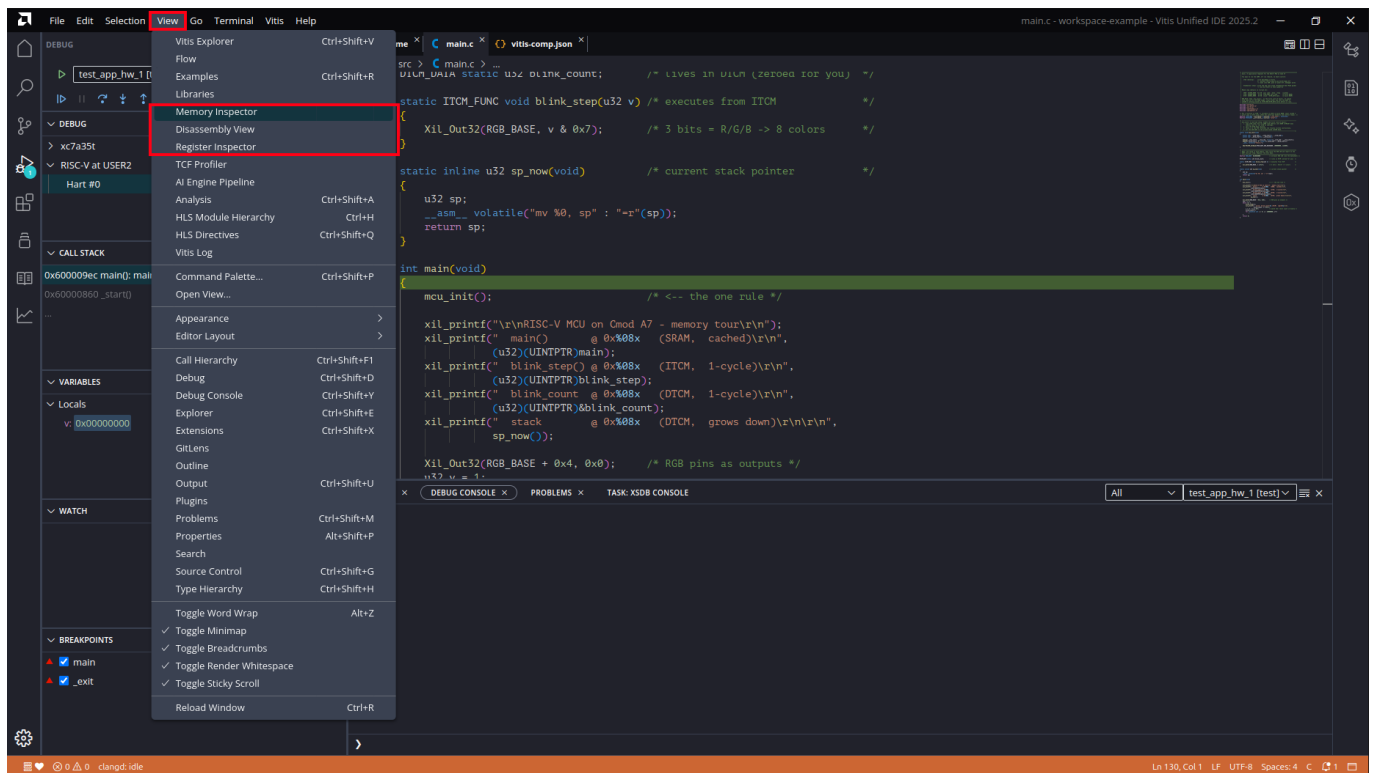
7.3 Breakpoints and Stepping

Click in the gutter next to a line number to set a breakpoint. The toolbar provides **Continue**, **Step Over**, **Step Into**, and **Step Out**. The left panel shows the call stack, variables, and watch expressions while the target is paused.

To run the application without the debugger, use **Run** in the FLOW panel instead — same loading, no pause at `main()`.

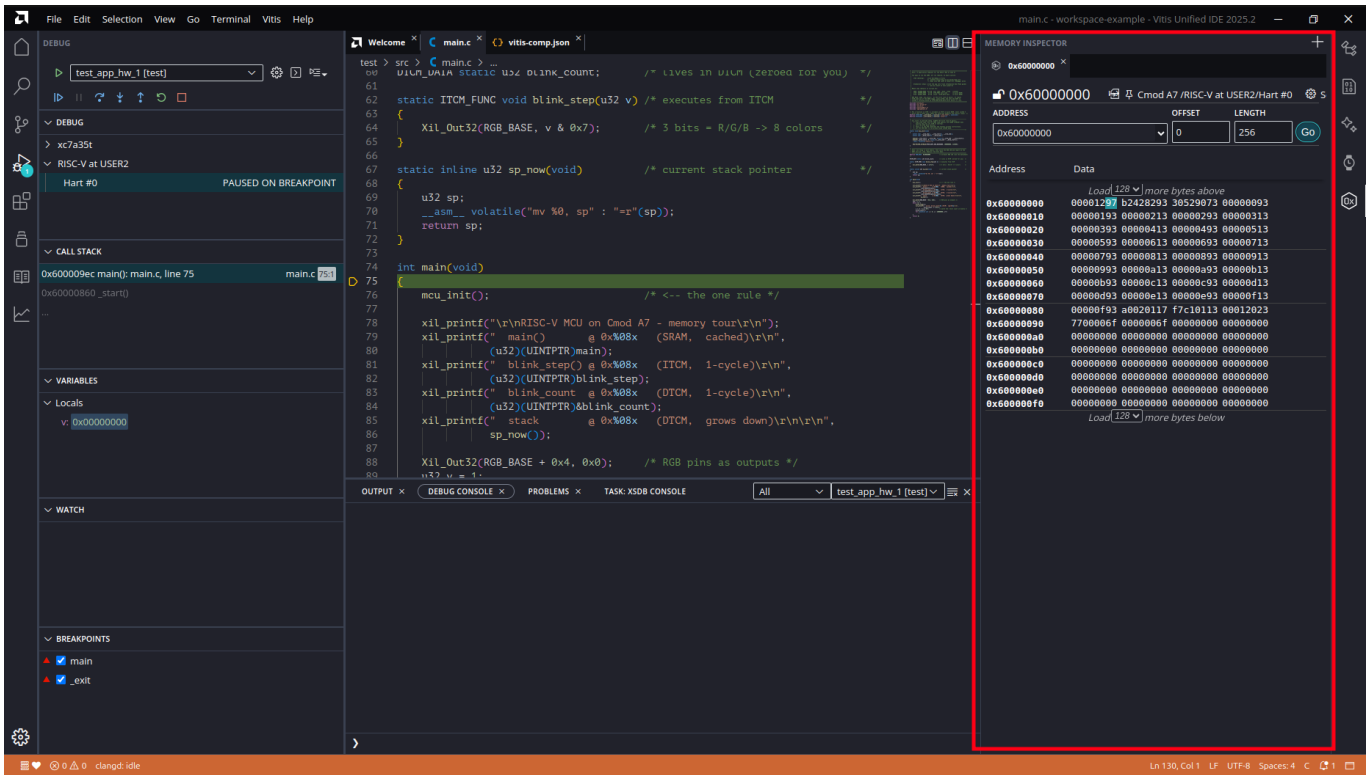
8. Inspection Tools

While a debug session is active, the **View** menu provides three inspectors:



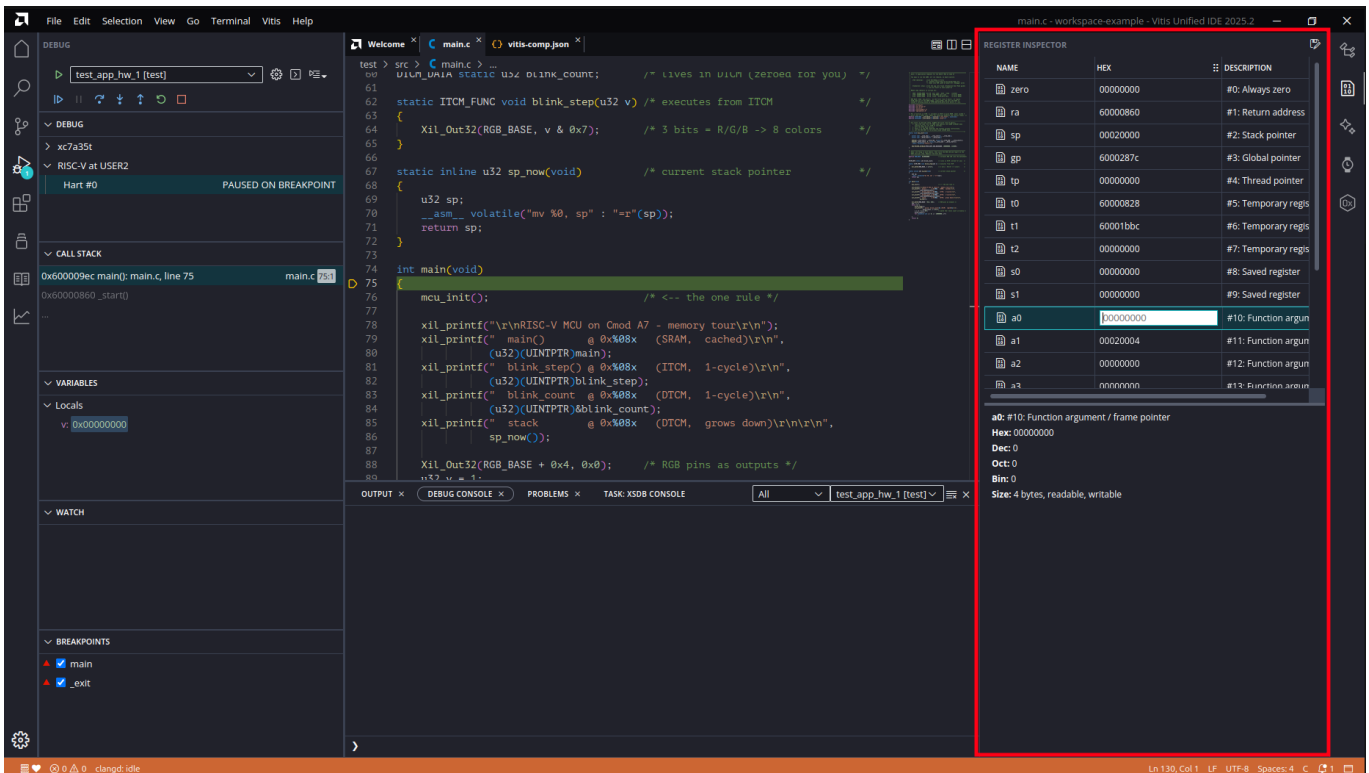
8.1 Memory Inspector

Enter an address to inspect memory directly — for example `0x60000000` (the application image in SRAM), `0x8000` (ITCM), or `0x10000` (DTCM).



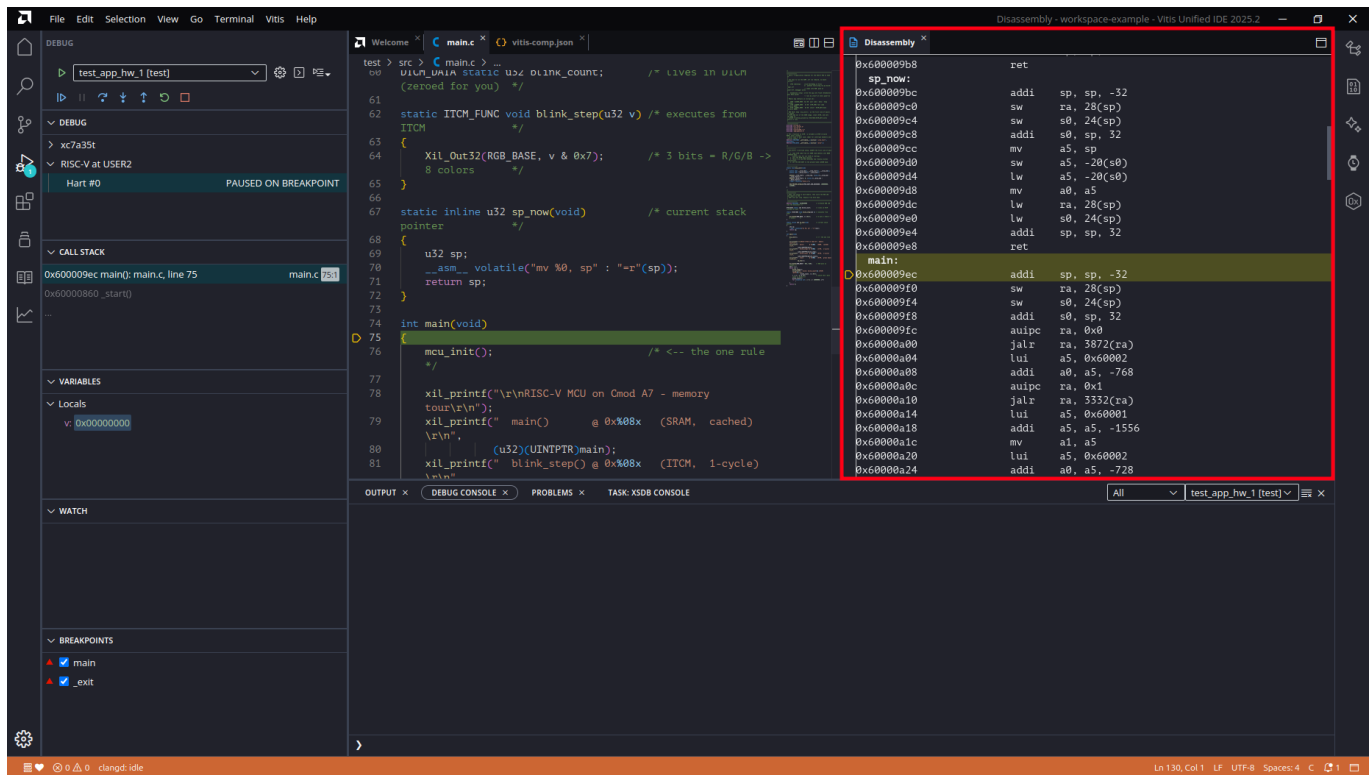
8.2 Register Inspector

Shows all processor registers with live values. With the target paused at `main()`, `sp` reads `0x00020000` — the top of the DTCM, matching the linker script's stack placement.



8.3 Disassembly View

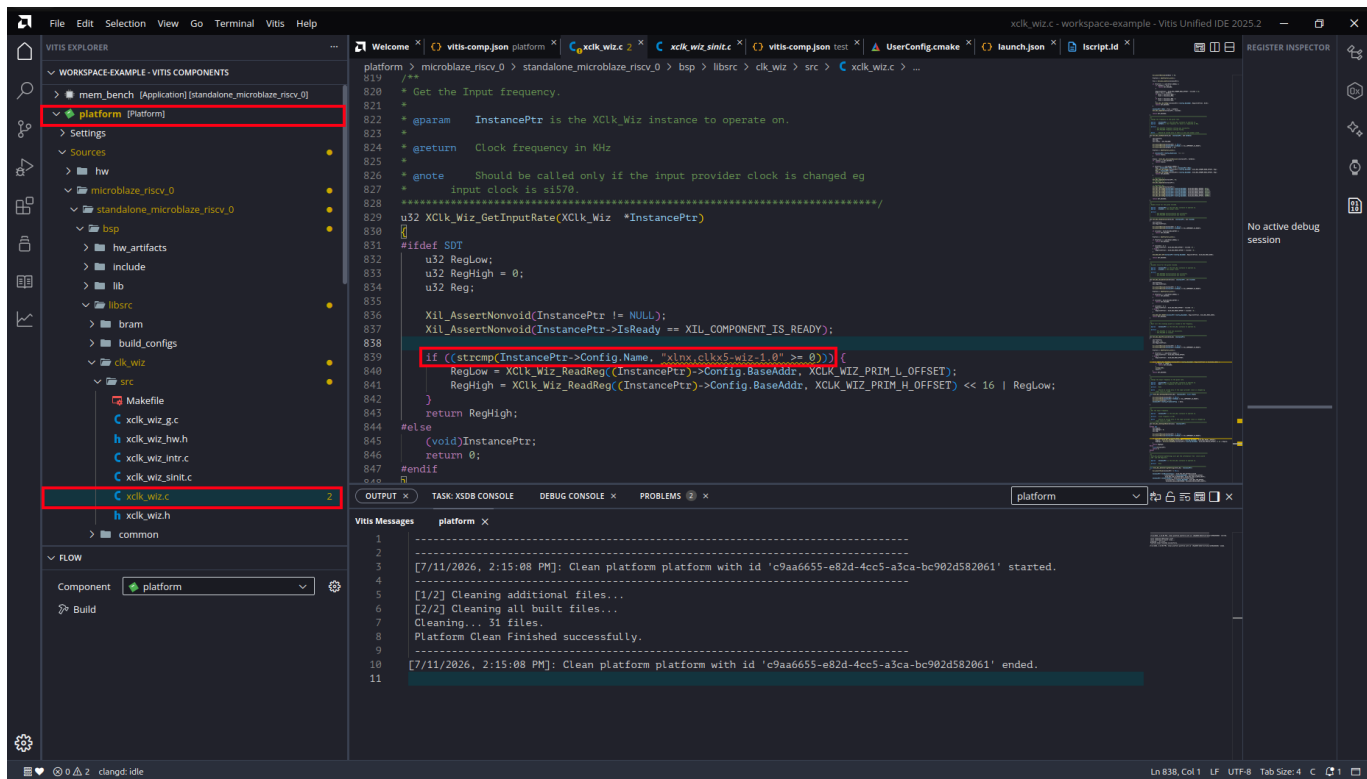
Shows the machine code actually executing, with source interleaving. The instruction addresses around `main()` all fall in `0x6000_xxxx` — the program runs from SRAM.



9. Troubleshooting

1. Platform build fails with **error: passing argument 2 of 'strcmp' ... [-wint-conversion]** in `xcclk_wiz.c` (reported as *"Error in generating platform"*). Two things combine to cause this:
2. The clock-management driver source that ships with Vitis 2025.2 (`clk_wiz` v1_10) contains a misplaced parenthesis in `xcclk_wiz.c` line 839.
3. If another RISC-V toolchain with GCC 14 or newer is on your PATH (for example one installed at `/opt/riscv` for assembly coursework), the BSP build may pick it instead of the Vitis-bundled compiler. GCC 14+ treats this construct as an error; the bundled compiler treats it as a warning.

The offending line, as the IDE shows it:

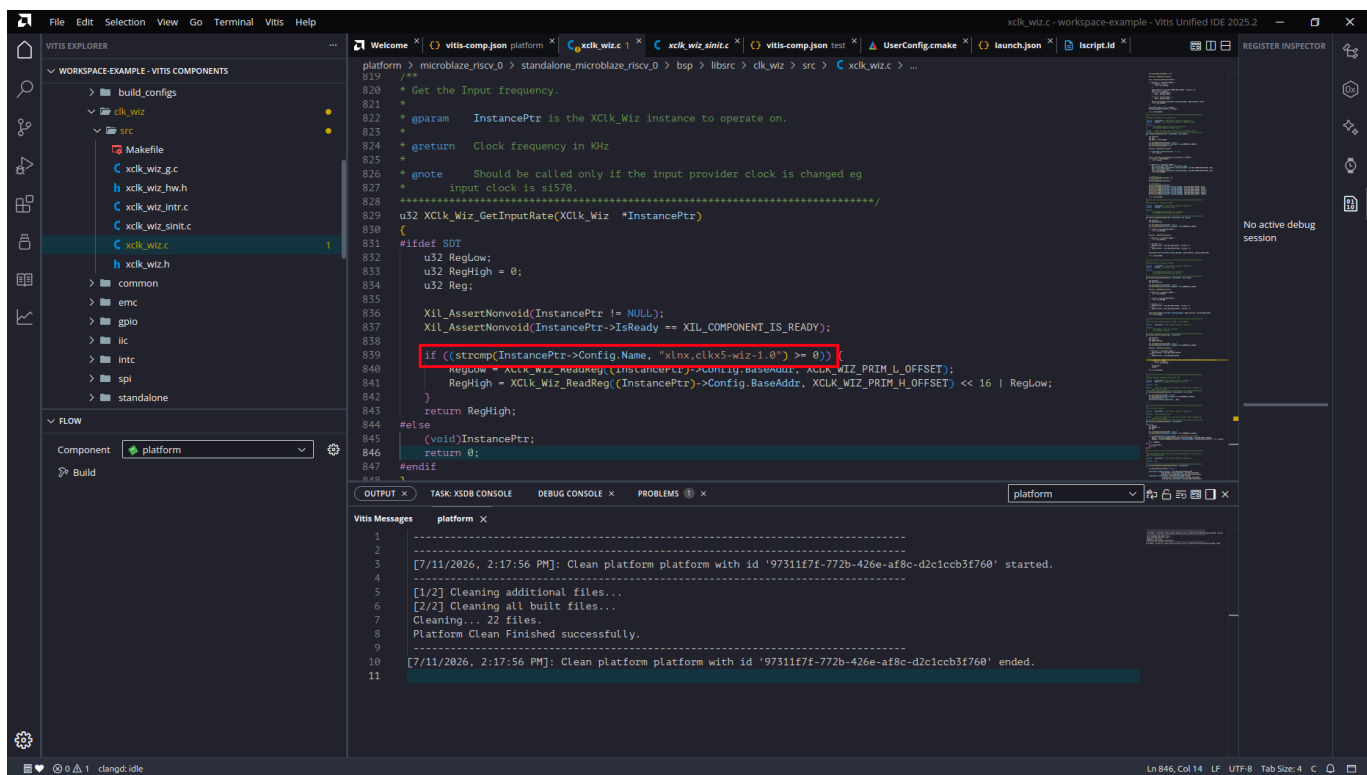


Fix the source (works with either compiler): change line 839 from

```
c strcmp(InstancePtr->Config.Name, "xlnx,clkx5-wiz-1.0" >= 0)
```

to

```
c strcmp(InstancePtr->Config.Name, "xlnx,clkx5-wiz-1.0") >= 0
```



Then rebuild the platform. Apply the same one-line fix to the Vitis installation copy at

```
/opt/Xilinx/2025.2/data/embeddedsw/XilinxProcessorIPLib/drivers/clk_wiz_v1_10/src/xclk_w
```

`iz.c` — the platform re-imports driver sources whenever the BSP is regenerated, so an unpatched installation brings the error back.

To check which compiler configured the BSP, read

`platform/.../bsp/libsrc/build_configs/gen_bsp/compile_commands.json`. To force a re-pick, delete `platform/.../bsp/libsrc/build_configs` and rebuild from a session whose PATH does not contain the other toolchain.

1. **The application runs (RGB LED cycles) but the USB console is silent** — `xil_printf` is going to the DIP-header UART. The BSP's stdout has reverted to `uart_1`: redo section 5, then rebuild the platform **and** the application. The compiled truth is

`platform/export/platform/sw/standalone_microblaze_riscv_0/include/bspconfig.h`:
`STDOUT_BASEADDRESS` must read `0x40300000` (`uart_USB`), not `0x40310000` (`uart_1`).